

Schreiben Sie einige Zeilen Java-Code, die ein **Bildschirm**-Objekt erzeugen und dann seine Methode **loeschen** nur genau dann aufrufen, wenn die Anzahl der Bildpunkte größer als 2 Millionen ist (kümmern Sie sich nicht darum, ob das logisch ist – das Ziel ist lediglich, dass etwas syntaktisch Korrektes aufgeschrieben wird, also etwas, das sich von einem Compiler übersetzen lassen könnte).

Übung 3.56 Beschreiben Sie die Veränderungen, die an der **Uhrenanzeige**-Klasse vorgenommen werden müssten, um Stunden, Minuten und Sekunden anzuzeigen. Wie viele **Nummernanzeige**-Objekte würde ein **Uhrenanzeige**-Objekt benötigen?

Übung 3.57 Schreiben Sie den Code für die Methode **taktsignalGeben** in **Uhrenanzeige**, die Stunden, Minuten und Sekunden anzeigt – oder implementieren Sie sogar die ganze Klasse, wenn Sie möchten.

Übung 3.58 Diskutieren Sie, ob das aktuelle Design der **Uhrenanzeige**-Klasse die Anzeige von Stunden, Minuten, Sekunden, Zehntelsekunden und Hundertstelsekunden unterstützen würde. Wie sähe beispielsweise der Code für die **taktsignalGeben**-Methode in diesem Fall aus?

Übung 3.59 *Zusatzaufgabe.* Im aktuellen Design von **Uhrenanzeige** ist ein **Uhrenanzeige**-Objekt dafür verantwortlich festzustellen, wann ein **Nummernanzeige**-Objekt auf null zurückgesprungen ist, und dann ein anderes **Nummernanzeige**-Objekt zu inkrementieren. Mit anderen Worten, es gibt keine direkte Verbindung zwischen **Nummernanzeige**-Objekten. Wäre es möglich, dass ein **Nummernanzeige**-Objekt einem anderen mitteilt, wenn es auf null zurückgesprungen ist, und dass sich dieses andere **Nummernanzeige**-Objekt dann hochzählt? Dann würde zum Beispiel das **minuten**-Objekt dem **stunden**-Objekt mitteilen, dass eine weitere Stunde vergangen ist, oder das **sekunden**-Objekt sagt dem **minuten**-Objekt, dass weitere sechzig Minuten verstrichen sind. Mit welchem dieser Objekte müsste die **taktsignalGeben**-Methode interagieren? Welche Datenfelder hätte ein **Nummernanzeige**-Objekt? Was sollte das **stunden**-Objekt tun, wenn ein ganzer Tag abgelaufen ist? Diskutieren Sie die Probleme dieses alternativen Designs und – wenn Ihnen der Sinn nach einer Herausforderung steht – versuchen Sie, es zu implementieren! Beachten Sie, dass Sie dazu eventuell etwas über das Java-Schlüsselwort **null** herausfinden müssen, das wir erst am Ende von Kapitel 4 behandeln.

KAPITEL

4

Objektsammlungen



Lernziele

Zentrale Konzepte in diesem Kapitel: Sammlungen, Iteratoren, Schleifen

Java-Konstrukte in diesem Kapitel: `ArrayList`, `Iterator`, `for-each`-Schleife, `while`-Schleife, `null`, anonyme Objekte

In diesem Kapitel fokussieren wir auf die Einführung verschiedener Möglichkeiten, Objekte in Sammlungen zusammenzufassen. Insbesondere werden wir die Klasse `ArrayList` als ein Beispiel für Sammlungen mit flexibler Größe diskutieren. Eng verbunden mit Sammlungen ist der Bedarf, über alle Elemente einer Sammlung zu iterieren. Zu diesem Zweck stellen wir zwei neue Kontrollstrukturen vor: die `for`-Schleife und die `while`-Schleife.

Dieses Kapitel ist lang und wichtig. Sie werden vermutlich nur dann ein erfolgreicher Programmierer, wenn Sie den Inhalt dieses Kapitels voll und ganz verstanden haben. Sie werden wahrscheinlich etwas länger für das Durcharbeiten benötigen als bei den vorherigen Kapiteln. Doch erliegen Sie nicht der Versuchung, das Kapitel nur oberflächlich zu überfliegen; nehmen Sie sich die Zeit und studieren Sie es sorgfältig.

4.1 Themen aus Kapitel 3 vertiefen

Wir werden in diesem Kapitel nicht nur neue Themen wie Sammlungen und Iteration einführen, sondern auch erneut auf zwei wichtige Themen aus Kapitel 3 eingehen: *Abstraktion* und *Objektinteraktion*. Wir haben gelernt, dass die Abstraktion uns erlaubt, ein Problem zu vereinfachen, indem wir es in einzelne Komponenten zerlegen. So können wir jede Komponente getrennt für sich betrachten, ohne uns in den Details eines großen Problems zu verlieren. Wir werden dieses Prinzip in die Praxis umsetzen, wenn wir die *Bibliotheksklassen* nutzen, die für Java verfügbar sind. Diese Klassen sind zwar streng genommen nicht Teil der Sprache, doch einige von ihnen sind so eng mit dem Schreiben von Java-Programmen verwoben, dass sie oft dafür gehalten werden. Deshalb prüfen Java-Programmierer normalerweise immer erst die Bibliotheken, um zu sehen, ob bereits jemand zu ihrem Problem eine Klasse geschrieben hat, die ihnen von Nutzen sein könnte. Das spart viel Arbeit, die die Programmierer besser in andere Teile des Programms investieren. Das gleiche Prinzip gilt für die meisten anderen Programmiersprachen, zu denen es ebenfalls Bibliotheken mit nütz-

lichen Klassen gibt. Es macht sich also bezahlt, sich mit dem Inhalt der Bibliotheken und den geläufigsten Klassen vertraut zu machen. Was die Abstraktion so stark macht, ist, dass wir normalerweise nicht unbedingt (falls überhaupt) wissen müssen, wie die Klasse im Innern aussieht, um sie effektiv nutzen zu können.

Wenn wir eine Bibliotheksklasse verwenden, folgt daraus, dass wir Code schreiben, der Instanzen dieser Klasse erzeugt, und anschließend interagieren unsere Objekte mit denen der Bibliotheksklasse. Deshalb spielt die Objektinteraktion in diesem Kapitel eine so große Rolle.

Sie werden feststellen, dass die Kapitel in diesem Buch immer wieder Themen aus vorherigen Kapiteln aufgreifen und vertiefen. Wir bezeichnen dies im Vorwort als „iterativen Ansatz“. Ein besonderer Vorteil dieses Ansatzes ist, dass Sie damit Ihr Wissen in den jeweiligen Bereichen peu à peu festigen, während Sie das Buch durcharbeiten.

Schließlich werden wir in diesem Kapitel unsere Definition des Konzepts der Abstraktion erweitern. Wir werden lernen, dass Abstraktion nicht nur bedeutet, Details zu verbergen, sondern auch, die gemeinsamen Eigenschaften und Muster zu erkennen, die in einem Programm immer wieder auftauchen. Diese gemeinsamen Muster sind der Grund, warum wir eine bereits geschriebene Methode oder Klasse teilweise oder ganz in einer neuen Situation wiederverwenden können. Dies gilt vor allem, wenn wir es mit Sammlungen und Iteration zu tun haben.

4.2 Die Abstraktion Sammlung

Eine der Abstraktionen, mit denen wir uns in diesem Kapitel beschäftigen werden, ist das Konzept der *Sammlung*, d.h. die Vorstellung, Dinge so zu gruppieren, dass wir auf alle gemeinsam Bezug nehmen und sie zusammen verwalten können. Eine Sammlung kann groß sein (alle Studenten einer Universität), klein sein (die Kurse, die ein Student belegt) oder sogar leer sein (alle Bilder von Picasso, die mir gehören).

Briefmarken, Autogramme, Filmplakate, Schmuckstücke, Musik oder was auch immer Sie sammeln – es wird immer einige Funktionen geben, die Sie hin und wieder auf die Sammlung ausführen wollen. Sie könnten der Sammlung etwas *hinzufügen* wollen oder die Sammlung *verkleinern*, zum Beispiel wenn Sie etwas doppelt haben oder Geld für Neuerwerbungen benötigen. Vielleicht wollen Sie auch Ihre Sammlung *umsortieren* – zum Beispiel nach Erwerbsdatum oder Wert. Was wir hier beschreiben sind typische *Operationen* auf eine Sammlung.

Im Programmierkontext wird die Abstraktion Sammlung zu einer bestimmten Art von Klasse und die Operationen darauf zu den Methoden dieser Klasse. Eine bestimmte Sammlung (meine Musiksammlung) wäre dann eine Instanz der Klasse. Und die in einer Sammlungsinstanz gespeicherten Elemente wären ebenfalls Objekte.

Weitere Beispiele für Sammlungen, deren Beziehung zu Programmierprojekten etwas offensichtlicher ist, wären zum Beispiel:

- Elektronische Kalender speichern Terminnotizen zu Verabredungen, Konferenzen, Geburtstagen usw. Werden weitere Termine verabredet, dann kommen neue Notizen hinzu; und alte Notizen werden gelöscht, wenn Einzelheiten zu vergangenen Ereignissen nicht länger benötigt werden.

Konzept

Sammlungen:
Sammlungsobjekte sind Objekte, die eine beliebige Anzahl anderer Objekte enthalten können.

- Bibliotheken speichern Informationen über ihre Bücher und Zeitschriften. Der Katalog ändert sich, wenn neue Bücher angeschafft oder alte Bücher ausgemustert oder eingelagert werden.
- Universitäten verwalten Datensätze über die immatrikulierten Studenten. Jedes akademische Jahr kommen neue Datensätze zu der Sammlung hinzu, während die Datensätze derer, die die Uni verlassen, in eine Archiv-Sammlung verschoben werden. Häufig tritt der Fall ein, dass Teilmengen der Sammlung ausgegeben werden: alle Studenten, die im ersten Semester Informatik studieren, oder alle Studenten, die dieses Jahr am Abschlussexamen teilnehmen.

Die Anzahl der Elemente in einer Sammlung ändert sich von Zeit zu Zeit. Bisher haben wir noch keine Funktionen von Java kennengelernt, die es uns erlaubt, eine beliebige Anzahl von Elementen zusammenzufassen. Wir könnten beispielsweise eine Klasse mit sehr vielen individuellen Datenfeldern definieren, um eine feste, aber sehr große Anzahl an Elementen aufzunehmen. Allerdings benötigen Programme in der Regel eine allgemeinere Lösung. Eine gute Lösung würde weder voraussetzen, dass wir im Voraus wissen, wie viele Elemente die Sammlung enthalten soll, noch verlangen, dass wir eine Obergrenze für die Anzahl angeben müssen.

Deshalb beginnen wir unsere Erkundung der Java-Bibliothek mit einer Klasse, die die einfachste Form einer Objektsammlung darstellt: eine unsortierte, aber geordnete Liste flexibler Größe namens **ArrayList**. In den nächsten Abschnitten werden wir als Beispiel eine persönliche Musiksammlung verwalten, um zu zeigen, wie wir eine beliebige Anzahl von Objekten in nur einem Containerobjekt unterbringen.

4.3 Ein Verwaltungssystem für Musikdateien

Wir werden eine Klasse schreiben, die uns bei der Organisation unserer Musikdateien auf einem Computer unterstützt. Unsere Klasse wird die Dateiinformationen nicht wirklich speichern, sondern sie wird die Verantwortung an die Standardbibliotheksklasse **ArrayList** delegieren, die uns viel Arbeit abnimmt. Warum müssen wir dann überhaupt eine eigene Klasse schreiben? Wer mit Bibliotheksklassen arbeitet, sollte sich auf alle Fälle merken, dass diese nicht für ein spezielles Anwendungsszenario geschrieben wurden, denn es sind allgemeine Klassen. Beispielsweise könnte ein **ArrayList**-Objekt Studentendatensätze als Objekte speichern, während ein anderes Terminnotizen speichert. Das bedeutet, dass erst die Klassen, die wir für die Nutzung der Bibliotheksklassen schreiben, szenariospezifische Operationen bieten, zum Beispiel eine Datei abspielen, die in der Sammlung gespeichert ist, wenn wir es mit Musikdateien zu tun haben.

Der Einfachheit halber arbeitet die erste Version dieses Projekts nur mit den Dateinamen der einzelnen Musik-Tracks. Es gibt keine zusätzlichen Angaben zu Titel, Interpret, Abspielzeit etc. Das bedeutet, wir bitten **ArrayList** lediglich, **String**-Objekte zu speichern, die Dateinamen repräsentieren. Indem wir in diesem Stadium die Dinge möglichst einfach halten, vermeiden wir, dass die Schlüsselkonzepte, die wir veranschaulichen möchten, in den Hintergrund gedrängt werden: die Erzeugung und die Nutzung eines Sammlungsobjekts. Später im Kapitel werden wir

unser Musiksammlung-Beispiel zu einem immer anspruchsvolleren Verwaltungs- und Abspielsystem ausbauen.

Wir gehen davon aus, dass jede Musikdatei einen Musik-Track repräsentiert. Die Beispieldateien zu diesem Projekt enthalten im Dateinamen den Namen des Interpreten und den Titel des Tracks – was wir uns später zunutze machen werden. Die erste Version unseres Verwaltungssystems, die wir als Ausgangsbasis nutzen, unterstützt folgende grundlegenden Operationen:

- Es können Tracks zu der Sammlung hinzugefügt werden.
- Es gibt keine Obergrenze bezüglich der Anzahl der zu speichernden Tracks (es sei denn durch die Speicherkapazität des Rechners).
- Wir erfahren, wie viele Tracks in der Sammlung sind.
- Es werden alle Tracks aufgelistet.

Es zeigt sich, dass die Klasse `ArrayList` es uns sehr leicht macht, diese Funktionalität aus unserer eigenen Klasse heraus zu bieten.

Beachten Sie, dass wir in dieser ersten Version nicht allzu ehrgeizig sind. Diese Operationen veranschaulichen lediglich die Grundlagen der Erzeugung und Verwendung der Klasse `ArrayList`. In späteren Versionen werden wir dann nach und nach weitere Funktionen hinzufügen, bis wir eine etwas anspruchsvollere Version haben. (Am wichtigsten wird es vielleicht sein, später die Möglichkeit vorzusehen, Musikdateien abzuspielen. Unsere erste Version ist dazu nicht in der Lage.) Dieser bescheidene inkrementelle Ansatz ist viel Erfolg versprechender als zu versuchen, alles auf einmal zu implementieren.

Bevor wir den Quelltext analysieren, der benötigt wird, um eine solche Klasse nutzen zu können, ist es sinnvoll, das Startverhalten des Musik-Verwaltungssystems zu untersuchen.

Übung 4.1 Öffnen Sie in BlueJ das Projekt *Musiksammlung-v1* und erzeugen Sie ein Objekt `MusikSammlung`. Speichern Sie darin die Namen einiger Audiodateien – dies sind einfache Zeichenketten. Da wir in diesem Stadium die Dateien noch nicht abspielen, sind beliebige Dateinamen möglich. Es gibt jedoch im *audio*-Ordner des Kapitels eine Reihe von Audiodateien, die Sie nutzen können.

Prüfen Sie, dass die Anzahl der Dateien, die von `gibAnzahlDateien` zurückgeliefert wird, mit der Anzahl übereinstimmt, die Sie gespeichert haben. Wenn Sie die Methode `dateiAusgeben` verwenden, müssen Sie einen Parameterwert von 0 (null) verwenden, um die erste Datei auszugeben, 1 (eins), um die zweite auszugeben, usw. Wir werden die Gründe für diese Form der Nummerierung zu gegebener Zeit erläutern.

Übung 4.2 Was passiert, wenn Sie ein neues `MusikSammlung`-Objekt erzeugen und dann `entferneDatei(0)` aufrufen, bevor Sie irgendwelche Dateien hinzugefügt haben? Erhalten Sie eine Fehlermeldung? Erwarten Sie einen Fehler?

Übung 4.3 Erzeugen Sie ein `MusikSammlung`-Objekt und fügen Sie zwei Dateinamen hinzu. Rufen Sie `dateiAusgeben(0)` und `dateiAusgeben(1)` auf, um die beiden Dateien auszugeben. Rufen Sie jetzt `entferneDatei(0)` und dann `dateiAusgeben(0)` auf. Was passiert? Entspricht dies Ihren Erwartungen? Können Sie erklären, was passiert sein könnte, als Sie den Dateinamen aus der Sammlung entfernt haben?

4.4 Eine Bibliotheksklasse verwenden

Listing 4.1 zeigt die volle Definition unserer Klasse `MusikSammlung`, die sich der Bibliotheksklasse `ArrayList` bedient. Beachten Sie, dass BlueJ die Bibliotheksklassen nicht im Klassendiagramm anzeigt.

Klassenbibliotheken

Objektorientierte Sprachen sind unter anderem deshalb so mächtig, weil sie oft von *Klassenbibliotheken* begleitet werden. Diese Bibliotheken enthalten normalerweise viele Hunderte oder Tausende von verschiedenen Klassen, die sich bereits in zahlreichen Projekten als sehr nützlich für die Entwickler erwiesen haben. In Java heißen diese Bibliotheken *Pakete*. Bibliotheksklassen werden genauso verwendet wie eigene Klassen. Instanzen werden mit `new` erzeugt und die Klassen haben Datenfelder, Konstruktoren und Methoden.

```
import java.util.ArrayList;

/**
 * Eine Klasse zur Verwaltung von Audiodateien.
 *
 * @author David J. Barnes und Michael Kölling.
 * @version 2016.02.29
 */
public class MusikSammlung
{
    // Eine ArrayList, in der die Namen von Audiodateien gespeichert werden können.
    private ArrayList<String> dateien;

    /**
     * Erzeuge eine MusikSammlung.
     */
    public MusikSammlung()
    {
        dateien = new ArrayList<>();
    }

    /**
     * Füge der Sammlung eine Datei hinzu.
     * @param dateiname die hinzuzufügende Datei
     */
    public void dateiHinzufuegen(String dateiname)
    {
        dateien.add(dateiname);
    }
}
```

Listing 4.1

Die Klasse
`MusikSammlung`.

```

/**
 * Liefere die Anzahl der Dateien in dieser Sammlung.
 * @return die Anzahl der Dateien in dieser Sammlung
 */
public int gibAnzahlDateien()
{
    return dateien.size();
}

/**
 * Gib eine Datei aus der Sammlung auf die Konsole aus.
 * @param index der Index der Datei, deren Name ausgegeben werden soll
 */
public void dateiAusgeben(int index)
{
    if(index >= 0 && index < dateien.size()) {
        String dateiname = dateien.get(index);
        System.out.println(dateiname);
    }
}

/**
 * Entferne eine Datei aus der Sammlung.
 * @param index der Index der zu entfernenden Datei
 */
public void entferneDatei(int index)
{
    if(index >= 0 && index < dateien.size()) {
        dateien.remove(index);
    }
}
}

```

4.4.1 Eine Bibliotheksklasse importieren

Die erste Zeile illustriert, wie mit einer *import-Anweisung* in einem Java-Programm der Zugriff auf Bibliotheksklassen deklariert wird:

```
import java.util.ArrayList;
```

Diese Anweisung macht die Klasse **ArrayList** aus dem Paket **java.util** innerhalb unserer Klassendefinition verfügbar. **import**-Anweisungen müssen immer vor den Klassendefinitionen in einer Datei stehen. Sobald eine Klasse auf diese Weise importiert wurde, kann sie wie eine selbst definierte Klasse verwendet werden. Dies tun wir hier, indem wir ein Datenfeld **dateien** definieren:

```
private ArrayList<String> dateien;
```

Hier lernen wir mit **String** in spitzen Klammern ein neues Konstrukt kennen: **<String>**. Auf den Grund dafür wurde bereits in Abschnitt 4.3 kurz angespielt, als wir erwähnten, dass **ArrayList** eine allgemeine Sammlungsklasse ist (d.h. eine Klasse, deren Speicherkapazität nicht beschränkt ist). Wenn wir eine **ArrayList**-Instanz erzeugen, müssen wir jedoch genau angeben, welche Art von Objekten in dieser speziellen Instanz gespeichert werden soll. Wir können jeden beliebigen Typ speichern, müssen aber diesen Typ angeben, wenn wir eine **ArrayList**-Variable deklarieren. Klassen wie **ArrayList**, die mit einem zweiten Typ parametrisiert werden, werden *generische Klassen* genannt. Später werden wir hierauf noch näher eingehen.

Wenn wir Sammlungen benutzen, müssen wir immer zwei Typen angeben: den Typ der Sammlung selbst (hier: **ArrayList**) und den Typ der Elemente, die wir in der Sammlung speichern wollen (hier: **String**). Wir können die gesamte Typdefinition

`ArrayList<String>` lesen als *Array-Liste von Strings*. Wir benutzen diese Typdefinition als den Typ für unser Datenfeld `dateien`.

Wie Sie inzwischen erwarten dürften, gibt es eine enge Verbindung zwischen dem Rumpf des Konstruktors und den Datenfeldern der Klasse, da der Konstruktor für die Initialisierung der Datenfelder jeder Instanz zuständig ist. Wie also die Klasse `Uhrenanzeige` für ihre beiden Datenfelder `Nummernanzeige`-Objekte erzeugte, sehen wir hier den Konstruktor der Klasse `MusikSammlung`, der ein Objekt vom Typ `ArrayList` erzeugt und in seinem Datenfeld `dateien` speichert.

4.4.2 Diamant-Notation

Bei der Erzeugung der `ArrayList`-Instanz haben wir die folgende Anweisung geschrieben:

```
dateien = new ArrayList<>();
```

Dies ist die sogenannte Diamant-Notation (weil die beiden spitzen Klammern eine diamantähnliche Form bilden), sie sieht ein wenig ungewöhnlich ein. Wir hatten vorher gesehen, dass die `new`-Anweisung die folgende Form hat:

```
new Typname ( Parameter )
```

und außerdem, dass der vollständige Typname für unsere Sammlung

```
ArrayList<String>
```

lautet. Daher sollte die Anweisung zum Erzeugen eines neuen Sammlungsobjekts mit einer leeren Parameterliste folgendermaßen aussehen:

```
dateien = new ArrayList<String>();
```

In der Tat würde diese Anweisung auch funktionieren. Die erste Version, die die Diamant-Notation verwendet (und somit den `String`-Typ nicht nennt), ist eher eine praktische Abkürzung. Wird die Erzeugung des Sammlungsobjekts mit einer Zuweisung kombiniert, dann erkennt der Compiler den Typ der Sammlungselemente anhand des Variablentyps auf der linken Seite der Zuweisung. Java erlaubt uns auch hier wieder, die Definition wegzulassen. Der Elementtyp wird automatisch aus dem Variablentyp gefolgert.

Die Verwendung dieser Form ändert nichts an der Tatsache, dass das zu erzeugende Objekt nichts anderes als `String`-Objekte speichern kann. Es dient also lediglich der Bequemlichkeit, weil es uns etwas Tipperei erspart und unseren Code verkürzt.

4.4.3 Schlüsselmethoden von ArrayList

Die Klasse `ArrayList` definiert etliche Methoden; wir werden an dieser Stelle jedoch nur vier von ihnen benutzen: `add`, `size`, `get` und `remove`.

Die Bedeutung der ersten beiden wird durch die Benutzung in den Methoden `dateiHinzufuegen` und `gibAnzahlDateien` verdeutlicht. Mit der Methode `add` an einer `ArrayList` wird der Liste ein Objekt hinzugefügt, die Methode `size` liefert die Anzahl der bereits gespeicherten Elemente. Wir werden in Abschnitt 4.7 betrach-

ten, wie die Methoden `get` und `remove` funktionieren, obwohl Sie wahrscheinlich schon vorher eine Vorstellung davon bekommen werden, wenn Sie den Code in den Methoden `dateiAusgeben` und `entferneDatei` lesen.

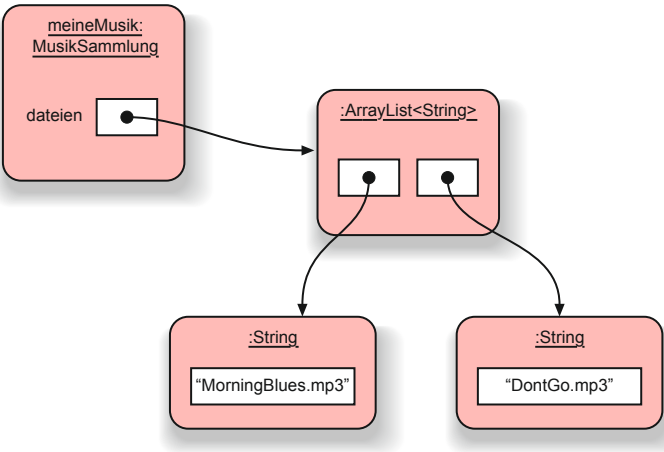
4.5 Objektstrukturen mit Sammlungen

Um besser verstehen zu können, wie ein Sammlungsobjekt der Klasse `ArrayList` arbeitet, sollten wir ein Objektdiagramm betrachten. Abbildung 4.1 zeigt, wie ein Objekt der Klasse `MusikSammlung` aussehen könnte, wenn bereits zwei Dateinamen gespeichert wurden. Vergleichen Sie Abbildung 4.1 mit Abbildung 4.2, in der ein dritter Dateiname abgelegt ist.

Es gibt mindestens drei Eigenschaften der Klasse `ArrayList`, die Ihnen auffallen sollten:

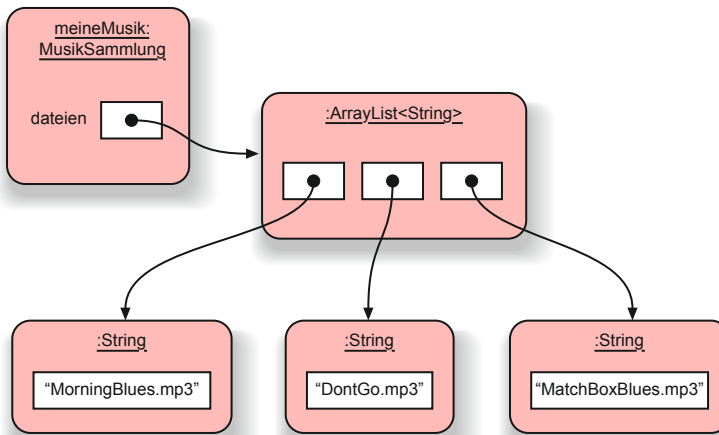
- Sie kann ihre interne Kapazität bei Bedarf vergrößern: Wenn weitere Objekte eingefügt werden, wird für diese einfach Platz geschaffen.
- Sie führt Buch über die Anzahl der aktuell gespeicherten Elemente. Die Methode `size` liefert diese Anzahl.
- Sie behält die Reihenfolge bei, in der die Elemente eingefügt werden. Die `add`-Methode speichert jedes neue Element am Ende der Liste. Später können die Elemente in der gleichen Reihenfolge wieder aus der Liste herausgeholt werden.

Abbildung 4.1
Eine Musiksammlung
mit zwei Dateien.



Wir stellen fest, dass die Klasse `MusikSammlung` sehr einfach aussieht – sie hat nur ein Datenfeld, in dem ein Objekt vom Typ `ArrayList<String>` gespeichert ist. Der schwierige Teil scheint durch das `ArrayList`-Objekt erledigt zu werden. Dies ist einer der großen Vorteile von Bibliotheksklassen: Jemand anderes hat Zeit und Aufwand investiert, um etwas Nützliches zu implementieren, und wir bekommen quasi kostenlosen Zugriff auf diese Funktionalität, indem wir die Klasse benutzen.

An diesem Punkt brauchen wir uns noch nicht darum zu kümmern, wie `ArrayList` diese Funktionalität implementiert. Es reicht uns, wenn wir diese nützliche Klasse schlicht benutzen können. Erinnern wir uns: Diese Ausblendung der Details ist ein Vorteil, den wir der Abstraktion verdanken: Es bedeutet, dass wir `ArrayList` zur Implementierung von beliebig vielen Klassen verwenden können, die eine Speicher-möglichkeit für eine nicht festgelegte Anzahl an Objekten benötigen.

**Abbildung 4.2**

Eine Musiksammlung mit drei Dateien.

Die zweite Eigenschaft – dass ein **ArrayList**-Objekt einen eigenen Zähler der eingefügten Elemente hält – hat wichtige Konsequenzen für die Art, wie wir die Klasse **MusikSammlung** implementieren. Obwohl eine Musiksammlung eine Methode **gibAnzahlDateien** hat, haben wir kein eigenes Datenfeld definiert, um diese Information zu speichern. Stattdessen *delegiert* eine Musiksammlung die Aufgabe, die Anzahl der eingefügten Elemente im Auge zu behalten, an sein **ArrayList**-Objekt. Auf diese Weise dupliziert eine Musiksammlung nicht solche Informationen, die es auch von anderer Stelle bekommen kann. Wenn ein Benutzer die Musiksammlung befragt, wie viele Dateinamen sie enthält, dann reicht die Musiksammlung diese Frage an das **dateien**-Objekt weiter und liefert dann dessen Antwort zurück.

Duplizierung von Information oder Verhalten ist etwas, das wir so weit wie möglich vermeiden sollten. Duplizierung deutet meist auf verschwendeten Aufwand hin und kann zu Inkonsistenzen führen, wenn zwei Dinge, die eigentlich gleich sein sollten, aufgrund eines Fehlers nicht mehr gleich sind. In den hinteren Kapiteln wird uns das Thema Duplizierung noch häufiger beschäftigen.

4.6 Generische Klassen

Die oben neu eingeführte Notation mit den spitzen Klammern müssen wir uns noch etwas ausführlicher ansehen. Der Typ unseres Datenfeldes **dateien** wurde deklariert als

```
ArrayList<String>
```

Die Klasse selbst, die wir hier verwenden, heißt einfach **ArrayList**, aber sie erfordert die Angabe eines zweiten Typs als Parameter, wenn sie zur Deklaration von Datenfeldern oder anderen Variablen verwendet wird. Klassen, die einen solchen Parameter erfordern, heißen, wie gesagt, *generische Klassen*. Generische Klassen definieren, im Gegensatz zu allen bisher betrachteten Klassen, nicht einen einzelnen Typ in Java, sondern potenziell viele Typen. Die Klasse **ArrayList** beispielsweise kann benutzt werden, um eine *ArrayList von Strings*, eine *ArrayList von Personen*, eine *ArrayList von Rechtecken* oder eine *ArrayList von jeder anderen verfügbaren Klasse* zu definieren. Jede spezielle **ArrayList** ist ein eigener Typ, der

in der Deklaration von Datenfeldern, Parametern und Methodenergebnissen verwendet werden kann. Wir können beispielsweise die folgenden beiden Datenfelder definieren:

```
private ArrayList<Person> mitglieder;  
private ArrayList<Ticketautomat> meineAutomaten;
```

Diese Definitionen besagen, dass **mitglieder** eine **ArrayList** hält, in der **Person**-Objekte gespeichert werden können, während **meineAutomaten** eine **ArrayList** halten kann, die **Ticketautomat**-Objekte speichert. Beachten Sie, dass **ArrayList <Person>** und **ArrayList<Ticketautomat>** verschiedene Typen sind. Diese Datenfelder können einander nicht zugewiesen werden, obwohl ihre Typen von derselben Klasse abgeleitet wurden.

Übung 4.4 Schreiben Sie die Deklaration eines privaten Datenfeldes **bibliothek**, das eine **ArrayList** halten kann. Die Elemente der **ArrayList** sollen vom Typ **Buch** sein.

Übung 4.5 Schreiben Sie die Deklaration einer lokalen Variablen **cs101**, die eine **ArrayList** von **Student**-Objekten hält.

Übung 4.6 Schreiben Sie eine Deklaration eines privaten Datenfeldes **tracks**, um eine Sammlung von **Musiktrack**-Objekten zu speichern.

Übung 4.7 Schreiben Sie Zuweisungen an die Variablen **bibliothek**, **cs101** und **tracks** (die Sie in den drei vorangegangenen Übungen definiert haben), um die entsprechenden **ArrayList**-Objekte zu erzeugen. Schreiben Sie sie einmal mit Diamant-Notation und einmal ohne, indem Sie den vollständigen Typ angeben.

Generische Klassen werden für verschiedene Zwecke benutzt; wir werden im Lauf dieses Buches noch weitere kennenlernen. Vorläufig sind Sammlungen wie **ArrayList** und einige weitere, die wir uns bald näher ansehen, die einzigen generischen Klassen, mit denen wir umgehen müssen.

4.7 Nummerierung in Sammlungen

Als wir das *Musiksammlung*-Projekt in den ersten paar Übungen untersucht haben, haben wir festgestellt, dass wir für die Ausgabe und das Entfernen von Dateien aus der Sammlung Parameter beginnend mit dem Wert null angeben mussten. Der Grund für diese Anforderung ist, dass die Elemente in einer Sammlung vom Typ **ArrayList** eine implizite Nummerierung oder Position haben, die mit null beginnt. Die Position eines Elements in einer Sammlung wird üblicherweise sein *Index* genannt. Das erste eingefügte Element in einer Sammlung hat den Index 0, das zweite den Index 1 und so weiter. Abbildung 4.3 illustriert dieselbe Situation wie oben, ergänzt um Indexnummern im **ArrayList**-Objekt.

Das heißt, dass dabei das letzte Element in einer Sammlung den Index *größe-1* hat. Zum Beispiel hat in einer Liste von 20 Elementen das letzte den Index 19.

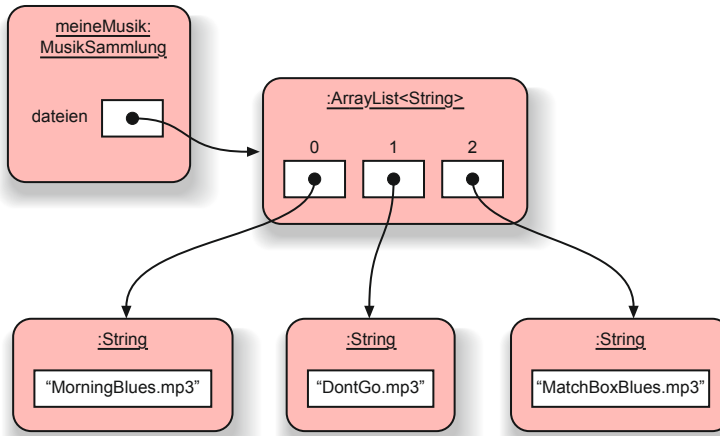


Abbildung 4.3
Indexnummern der
Elemente in einer
Sammlung.

Die Methoden `dateiAusgeben` und `entferneDatei` veranschaulichen beide, wie Indexnummern verwendet werden, um Zugriff auf ein Element in einer **ArrayList** zu erhalten: im ersten Fall über ihre `get`-Methode und im zweiten über ihre `remove`-Methode. Beachten Sie, dass beide Methoden sicherstellen, dass ihr Parameter im Bereich der gültigen Indexwerte `[0...size()-1]` liegt, bevor sie den Index in die **ArrayList**-Methoden hineinreichen. Diese Validierung ist guter Programmierstil, den Sie möglichst übernehmen sollten, da er verhindert, dass der Aufruf einer Bibliotheksklassenmethode aufgrund potenziell ungültiger Parameter fehlschlägt.

Fallstrick

Bei nachlässiger Benutzung kann es Ihnen passieren, dass Sie einen Zugriff auf ein Element mit einem Index außerhalb der gültigen Indexgrenzen einer **ArrayList** versuchen. In einem solchen Fall erhalten Sie eine Fehlermeldung. Diesen Fehler bezeichnen wir als *Index-Grenzverletzung* (*index-out-of-bounds*). In Java werden Sie entsprechend eine Meldung über eine **IndexOutOfBoundsException** zu sehen bekommen.

Übung 4.8 Wenn eine Sammlung 10 Elemente enthält, welchen Wert liefert dann ein Aufruf ihrer Methode `size`?

Übung 4.9 Schreiben Sie eine Methode, die mithilfe der Methode `get` das fünfte Objekt in einer Sammlung mit dem Namen `elemente` liefert.

Übung 4.10 Welchen Index hat das letzte Element in einer Sammlung von 15 Objekten?

Übung 4.11 Schreiben Sie eine Methode, die ein Objekt, auf das die Variable `lieblingstrack` verweist, in die Sammlung `dateien` einfügt.

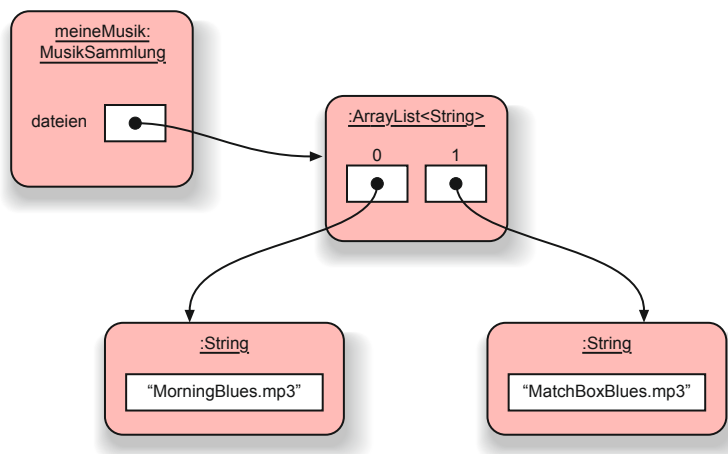
4.7.1 Elemente aus einer Sammlung entfernen

Nicht nur das Hinzufügen von Elementen zu einer Sammlung ist wichtig, sondern auch das Entfernen von Elementen, wie wir bereits anhand der Methode `entferneDatei` in Listing 4.1 sehen konnten. Hierfür bietet die Klasse `ArrayList` eine Methode `remove`, die den Index des zu entfernenden Elements als Parameter bekommt. Eine Besonderheit beim Entfernen von Elementen ist, dass es die Indexwerte der restlichen in der Sammlung gespeicherten Objekte ändert. Wenn ein Element mit einer niedrigen Indexnummer entfernt wird, dann verschiebt die Sammlung alle nachfolgenden Elemente um eine Position, um die Lücke zu füllen. Folglich werden ihre Indexnummern um 1 verringert.

Abbildung 4.4 illustriert, wie die Indexwerte einiger Elemente in einer `ArrayList` sich verschieben, wenn ein Element in der Mitte entfernt wird. Ausgehend von der Situation in Abbildung 4.3 wurde das Objekt mit der Nummer 1 entfernt. Als Ergebnis hat sich der Index des Objekts mit dem ursprünglichen Index 2 auf 1 geändert, während der Index des Objekts mit der Nummer 0 unverändert geblieben ist.

Zusätzlich werden wir später noch sehen, dass es neben dem Anfügen am Ende noch andere Möglichkeiten zum Einfügen von Elementen in eine `ArrayList` gibt. Dies bedeutet, dass sich bei Elementen, die bereits in der Sammlung enthalten sind, beim Einfügen eines neuen Elements ihre Indexnummern verschieben können. Den Benutzern sollten solche möglichen Änderungen der Indizes durch Einfügen oder Entfernen bewusst sein.

Abbildung 4.4
Änderung von
Indexnummern nach
Entfernen eines
Elements.



4.7.2 Vom allgemeinen Nutzen der Nummerierung (von Sammlungen)

Die Verwendung von ganzzahligen Indexwerten, um auf Objekte in einer Sammlung zuzugreifen, wird uns immer wieder begegnen – und zwar nicht nur in `ArrayList`-Instanzen, sondern auch in anderen Sammlungsstypen. Deshalb ist es so wichtig, dass wir alles, was wir bisher gesehen haben, auch verstehen: dass die Indexwerte bei null beginnen, dass die Objekte sequenziell durchnummeriert werden und dass es in der Regel keine Lücken in den Indexwerten der aufeinanderfolgenden Objekte in der Sammlung gibt.

Ganze Zahlen als Indexwerte zu verwenden, macht es einfach, im Programmcode Dinge wie „das nächste Element“ oder „das vorherige Element“ mit Bezug auf ein Element in der Sammlung auszudrücken. Wenn ein Element an Indexposition **p** steht, dann steht das nächste an Index (**p+1**) und das vorherige an Index (**p-1**). Wir können auch eine Auswahl von Elementen in natürlicher Sprache wie „die ersten drei“ bzw. „die Elemente 0, 1 und 2“ in einer Programmiersprache ausdrücken. Oder wir könnten „die letzten vier“ Elemente ausdrücken als die Elemente an den Indexpositionen „(**liste.size()-4**) bis (**liste.size()-1**)“. Wir könnten uns sogar vorstellen, eine ganze Sammlung durchzugehen, indem wir eine Indexvariable vom Typ **int** einrichten, deren Wert am Anfang auf null gesetzt und anschließend sukzessive um 1 erhöht wird. Dann übergeben wir ihren Wert der **get**-Methode, um auf jedes Element in der Liste der Reihe nach zuzugreifen, und hören auf, wenn der letzte Indexwert überschritten wurde.

Doch wir greifen vor. Wir werden in Bälde im Zusammenhang mit Schleifen und Iterationen sehen, wie all dies funktioniert.

Übung 4.12 Schreiben Sie eine Methode, die das dritte Objekt aus der Sammlung **verabredungen** entfernt.

Übung 4.13 Nehmen Sie an, dass in einer Sammlung ein Objekt bei Index 6 gespeichert ist. Welchen Index wird dieses Objekt haben, nachdem die Objekte bei Index 0 und Index 9 entfernt wurden?

Übung 4.14 Fügen Sie zu der Klasse **MusikSammlung** eine Methode **indexPruefen** hinzu. Diese Methode übernimmt einen **int**-Parameter und prüft, ob er ein gültiger Index für die aktuelle Sammlung ist. Um gültig zu sein, muss der Parameter im Bereich 0 bis **size()-1** liegen.

Wenn der Parameter nicht gültig ist, sollte die Methode eine Fehlermeldung ausgeben, die Auskunft über den gültigen Indexbereich gibt. Ist der Index gültig, sollte nichts ausgegeben werden. Testen Sie Ihre Methode auf der Objektleiste sowohl mit gültigen als auch ungültigen Parametern. Funktioniert Ihre Methode auch, wenn Sie den Index einer leeren Sammlung prüfen?

Übung 4.15 Schreiben Sie eine alternative Version von **indexPruefen** namens **gueltigerIndex**. Diese Methode soll einen **int**-Parameter übernehmen und einen booleschen Wert zurückliefern. Sie gibt nichts aus, sondern liefert **true** zurück, wenn der Wert des Parameters ein gültiger Index für die aktuelle Sammlung ist, und andernfalls **false**. Testen Sie Ihre Methode auf der Objektleiste sowohl mit gültigen als auch ungültigen Parametern. Testen Sie auch für den Fall, dass die Sammlung leer ist.

Übung 4.16 Ändern Sie die Methoden **dateiAusgeben** und **entferneDatei** in der Klasse **MusikSammlung** so, dass sie statt des booleschen Ausdrucks Ihre Methode **gueltigerIndex** verwenden, um ihren Parameter zu prüfen. Sie sollten nur dann **get** oder **remove** auf die **ArrayList** aufrufen, wenn **gueltigerIndex true** zurückliefert.

4.8 Musikdateien abspielen

Es wäre nett, wenn unser Verwaltungssystem nicht nur eine Liste von Musikdateien speichern würde, sondern zusätzlich die Möglichkeit bieten würde, Dateien abzuspielen. Auch hier kann uns die Abstraktion helfen. Wenn wir eine Klasse zur Verfügung hätten, die speziell dem Abspielen von Audiodateien dient, müsste unsere **MusikSammlung**-Klasse das nicht auch noch übernehmen. Sie würde lediglich den Namen der Datei in die Abspiel-Klasse reichen und ihr den Rest überlassen.

Leider gibt es in der Java-Standardbibliothek keine Klasse, mit der das von uns verwendete Audioformat *.mp3* abgespielt werden kann. Doch glücklicherweise gibt es viele Programmierer, die ständig eigene nützliche Klassen schreiben und sie anderen Personen zur Verfügung stellen. Diese werden oft als „Bibliotheken von Drittanbietern“ bezeichnet und genauso importiert und verwendet wie die Klassen der Standard-Java-Bibliothek.

Für die nächste Version unseres Projekts haben wir eine Reihe von Klassen aus **javazoom.net** herangezogen, um eine eigene Musikplayer-Klasse zu schreiben. Sie finden sie in der Version *Musiksammlung-v2*. Die drei Methoden der Klasse **MusikPlayer**, die wir verwenden, lauten **dateiAnspielen**, **starteAbspielen** und **stop**. Die ersten beiden nehmen den Namen der abzuspielenden Audiodatei entgegen. Die erste spielt einige Sekunden vom Anfang der Datei und kehrt zurück, wenn sie mit dem Abspielen fertig ist, während die zweite das Abspielen im Hintergrund startet und dann die Kontrolle sofort wieder an das Verwaltungssystem zurückgibt, was die Notwendigkeit für eine **stop**-Methode erklärt (falls Sie das Abspielen abbrechen wollen). Listing 4.2 enthält die neuen Elemente der Klasse **MusikSammlung**, die auf einige der Abspielfunktionen zugreifen.

Listing 4.2

Abspielfunktionen
der Klasse
MusikSammlung.

```
import java.util.ArrayList;

/**
 * Eine Klasse zur Verwaltung von Audiodateien.
 * Diese Version kann die Dateien auch abspielen.
 *
 * @author David J. Barnes und Michael Kölling.
 * @version 2016.02.29
 */
public class MusikSammlung
{
    // Eine ArrayList, in der die Namen von Audiodateien gespeichert werden können.
    private ArrayList<String> dateien;
    // Ein Player zum Abspielen der Musikdateien.
    private MusikPlayer player;

    /**
     * Erzeuge eine MusikSammlung.
     */
    public MusikSammlung()
    {
        dateien = new ArrayList<>();
        player = new MusikPlayer();
    }

    /**
     * Starte das Abspielen einer Datei aus der Sammlung.
     * Zum Beenden des Abspielvorgangs verwende beendeAbspielen() .
     * @param index der Index, der abzuspielenden Datei
     */
    public void starteAbspielenDatei(int index)
    {
        String dateiname = dateien.get(index);
        player.starteAbspielen(dateiname);
    }
}
```

```

/**
 * Stoppt den Player.
 */
public void beendeAbspielen()
{
    player.stop();
}

// weitere Details hier ausgelassen
}

```

Übung 4.17 Erzeugen Sie in der zweiten Version unseres Projekts ein Objekt der Klasse **MusikSammlung**. Experimentieren Sie ein wenig herum, indem Sie Dateien hinzufügen und abspielen.

Wenn Sie die Dateien nutzen wollen, die im Kapitelordner *audio* gespeichert sind, müssen Sie den Namen des Ordners zusammen mit dem Dateinamen und dem Suffix im **dateiname**-Parameter angeben. Um zum Beispiel die Datei *Blind-Blake-EarlyMorningBlues.mp3* aus dem *audio*-Ordner zu verwenden, müssen Sie der Methode **dateiHinzufuegen** den String **"../audio/BlindBlake-EarlyMorningBlues.mp3"** übergeben.

Sie können auch eigene *mp3*-Dateien verwenden. Sie müssen diese Dateien nur in dem *audio*-Ordner ablegen. Denken Sie jedoch daran, den Ordnernamen als Teil des Dateinamens anzugeben.

Experimentieren Sie auch mit Dateinamen, die nicht existieren. Was passiert dann?

4.8.1 Zusammenfassung des Musiksammlung-Projekts

Wir haben mit den grundlegenden Funktionen zur Organisation unserer Musiksammlung gute Fortschritte gemacht. Wir können die Namen einer beliebigen Anzahl von Musikdateien speichern und sogar abspielen. Und der Programmieraufwand war viel geringer als gedacht, da wir die Möglichkeit hatten, von den Funktionen der folgenden Bibliotheksklassen zu profitieren: **ArrayList** aus der Java-Standardbibliothek und eine Musikplayer-Klasse, die eine Klassenbibliothek eines Drittanbieters verwendet. Außerdem mussten wir kaum etwas von den internen Abläufen in diesen Bibliotheksklassen wissen. Es reichte, die Namen, Parametertypen und den Ergebnistyp der Schlüsselmethoden zu kennen.

Es fehlen jedoch noch einige wichtige Funktionen, wenn wir ein wirklich nützliches Programm implementieren wollen. Am auffälligsten ist, dass es beispielsweise gar keine Möglichkeit gibt, die ganze Sammlung auszugeben. Dies soll Thema des nächsten Abschnitts sein, in dem wir die erste von mehreren Java-Kontrollstrukturen einführen.

4.9 Komplette Sammlungen verarbeiten

Am Ende des letzten Abschnitts haben wir erwähnt, dass es nützlich wäre, eine Methode in der Klasse **MusikSammlung** zu haben, die alle Dateinamen der Sammlung ausgibt. Da wir wissen, dass jeder Dateiname der Sammlung mit einer eindeutigen Indexnummer verbunden ist, besteht eine Möglichkeit, die Ausgabe umzusetzen, darin, die Dateinamen, die bei aufeinanderfolgenden, zunehmenden Indexnummern gespeichert sind, beginnend mit null auszugeben. Bevor Sie weiterlesen, sollten Sie anhand der folgenden Übungen feststellen, ob Sie mit den Java-Mitteln, die wir bereits kennen, eine solche Methode schreiben können.

Übung 4.18 Wie würde der Kopf einer Methode **alleDateienAusgeben** in der Klasse **MusikSammlung** aussehen? Welchen Ergebnistyp sollte sie haben? Benötigt sie Parameter?

Übung 4.19 Wir wissen, dass das erste Element einer **ArrayList** bei Index 0 gespeichert ist und die Liste die Dateinamen als Zeichenketten speichert. Könnten wir den Rumpf der Methode **alleDateienAusgeben** ungefähr nach folgendem Muster schreiben?

```
System.out.println(dateien.get(0));
System.out.println(dateien.get(1));
System.out.println(dateien.get(2));
```

usw.

Wie viele **println**-Aufrufe wären notwendig, um die Methode zu vervollständigen?

Vermutlich haben Sie bemerkt, dass diese Frage nicht wirklich beantwortet werden kann, weil die Antwort von der Anzahl der Dateien abhängt, die zum Zeitpunkt der Ausgabe in der Liste enthalten sind. Bei drei Dateien würden wir drei **println**-Aufrufe benötigen, bei vier wären es vier usw. Die Methoden **dateiAusgeben** und **entferneDatei** zeigen, dass der Bereich der gültigen Indexnummern zu jedem Zeitpunkt von null bis zur Anzahl der zusammengetragenen Dateien minus eins definiert ist. Entsprechend sollte eine Methode **alleDateienAusgeben** ebenfalls die dynamische Größe der Liste berücksichtigen, um die Aufgabe korrekt auszuführen.

Wir haben es hier mit einer Situation zu tun, in der etwas mehrfach getan werden soll; aber wie oft genau, das hängt von Umständen ab, die variieren können – in diesem Fall der Größe der Sammlung. Diese Art von Problem werden wir bei der Programmierung sehr häufig haben, und die meisten Programmiersprachen bieten mithilfe von *Schleifenanweisungen* (die auch als *iterative Kontrollstrukturen* bezeichnet werden) mehrere Wege, um es zu lösen.

Die erste Schleife, die wir zur Ausgabe der Dateien einführen wollen, ist eine der besonderen Art für Sammlungen, da sie komplett auf eine Indexvariable verzichtet: die *for-each-Schleife*.

4.9.1 Die for-each-Schleife

Eine *for-each-Schleife* ist eine Möglichkeit, eine Menge von Aktionen auf den Elementen einer Sammlung zu wiederholen, ohne dass die Aktionen mehrfach in den Quelltext geschrieben werden müssen, wie wir dies in **Übung 4.19** gesehen haben. Die Struktur einer **for-each-Schleife** kann durch folgenden Pseudocode dargestellt werden:

```
for (Elementtyp element : sammlung) {
    Schleifenrumpf
}
```

Das entscheidend neue Java-Konzept an dieser Stelle ist das Schlüsselwort **for**. Die Sprache Java bietet zwei Varianten einer **for**-Schleife: Eine ist die *for-each-Schleife*, die wir hier gerade näher betrachten, die andere heißt einfach *for-Schleife* und wird in Kapitel 7 diskutiert.

Eine **for-each-Schleife** hat zwei Bestandteile: einen Schleifenkopf (die erste Zeile der Schleifenanweisung) und einen darauf folgenden Schleifenrumpf. Der Rumpf enthält die Anweisungen, die wir wiederholt ausführen wollen.

Die **for-each-Schleife** hat ihren Namen von der Art, wie wir sie lesen können („for each“ lässt sich übersetzen als „für jedes“): Wenn wir das Schlüsselwort **for** lesen als „für jedes“ und den Doppelpunkt als „in“, dann ergibt die Struktur im oben angegebenen Pseudocode sinngemäß Folgendes:

```
für jedes element in sammlung tue: {
    Schleifenrumpf
}
```

Wenn Sie diese Version mit der ersten vergleichen, dann sehen Sie, dass *element* oben in Form einer Variablendeklaration als **Elementtyp element** geschrieben wurde. Es wird auch tatsächlich eine Variable deklariert, die dann nacheinander für alle Elemente der Sammlung verwendet wird. Aber bevor wir das weiter ausführen, sollten wir uns zuerst ein Beispiel mit echtem Java-Code ansehen.

Listing 4.3 zeigt eine Implementierung einer Methode **alleDateienAusgeben**, die alle Dateien in der **ArrayList** der Musiksammlung mithilfe einer **for-each-Schleife** auf die Konsole ausgibt.

```
/**
 * Gib eine Liste aller Dateien in der Sammlung aus.
 */
public void alleDateienAusgeben()
{
    for(String dateiname : dateien) {
        System.out.println(dateiname);
    }
}
```

Konzept

Eine **Schleife** kann benutzt werden, um einen Block von Anweisungen wiederholt ausführen zu lassen, ohne dass die Anweisungen mehrfach in den Quelltext geschrieben werden müssen.

Listing 4.3

Benutzung einer **for-each-Schleife** zur Ausgabe aller Dateien.

In dieser **for-each-Schleife** wird der Schleifenrumpf – der aus der einzelnen Anweisung **System.out.println** besteht – wiederholt ausgeführt, einmal für jedes Element in der **ArrayList**-Sammlung **dateien**. Wenn beispielsweise vier Strings in der Liste wären, würde die Anweisung viermal ausgeführt.

Vor jeder Ausführung der Anweisung wird der Variablen **dateiname** eines der Elemente in der Liste zugewiesen, zuerst das bei Index 0, dann das bei Index 1 und so weiter. Auf diese Weise wird jedes Element in der Liste ausgegeben.

Lassen Sie uns diese Schleife noch etwas genauer ansehen. Das Schlüsselwort **for** leitet die Schleife ein. Es wird gefolgt von einem Paar runder Klammern, in denen die Details der Schleife definiert werden. Das erste Detail ist die Deklaration **String dateiname** – dies deklariert eine neue lokale Variable **dateiname**, die nacheinander die Elemente der Liste enthält. Wir nennen diese Variable die *Schleifenvariable*. Wir können den Namen dieser Variablen wie bei jeder Variablen frei wählen, es muss nicht wie hier „dateiname“ sein. Der Typ der Schleifenvariablen muss übereinstimmen mit dem deklarierten Typ der Elemente der Sammlung, die wir benutzen wollen; in unserem Fall ist er **String**.

Es folgen ein Doppelpunkt und die Variable, die die Sammlung hält, die wir verarbeiten wollen. Jedes Element dieser Sammlung wird nacheinander der Schleifenvariablen zugewiesen und nach jeder dieser Zuweisungen wird der Schleifenrumpf einmal ausgeführt. Im Schleifenrumpf können wir uns über die Schleifenvariable auf jedes Element beziehen.

Um zu überprüfen, ob Sie die Arbeitsweise dieser Schleife verstehen, sollten Sie die folgenden Übungen bearbeiten.

Übung 4.20 Implementieren Sie die Methode **alleDateienAusgeben** in Ihrer Version des *Musiksammlung*-Projekts. (Eine Lösung, in der diese Methode implementiert ist, finden Sie im Projekt *Musiksammlung-v3*, aber für einen besseren Lerneffekt empfehlen wir, dass Sie die Methode selbst implementieren.)

Übung 4.21 Erzeugen Sie eine **MusikSammlung** und speichern Sie einige Dateinamen darin. Benutzen Sie die Methode **alleDateienAusgeben**, um zu prüfen, ob diese wie gewünscht funktioniert.

Übung 4.22 Erzeugen Sie in der Direkteingabe ein **ArrayList<String>**-Objekt, indem Sie die folgenden zwei Zeilen eingeben:

```
import java.util.ArrayList;
new ArrayList<String>()
```

Wenn Sie die letzte Zeile nicht mit einem Semikolon abschließen, erscheint das kleine rote Objektsymbol. Ziehen Sie dieses Symbol in die Objektleiste. Informieren Sie sich über die zur Verfügung stehenden Methoden und versuchen Sie, einige davon aufzurufen (z.B. **add**, **remove**, **size**, **isEmpty**). Rufen Sie die gleichen Methoden über die Direkteingabe auf. Sie können aus der Direkteingabe auf die Objekte in der Objektleiste zugreifen, indem Sie deren Namen eingeben. Wenn Sie z.B. ein **ArrayList** über den Namen **all** auf die Objektleiste gelegt haben, können Sie es in der Direkteingabe ansprechen mit

```
all.size()
```

Übung 4.23 Wenn Sie wollen, können Sie den Debugger verwenden, um zu beobachten, wie die Anweisungen in der Schleife in `alleDateienAusgeben` wiederholt werden. Setzen Sie einen Haltepunkt unmittelbar vor die Schleife und gehen Sie die Methode Schritt für Schritt durch, bis die Schleife alle Elemente verarbeitet hat und die Methode endet.

Übung 4.24 Zusatzaufgabe. Die `for-each`-Schleife verwendet keine explizite `int`-Variable, um nacheinander auf die Elemente in der Liste zuzugreifen. Wenn wir also den Index von jedem Dateinamen mit ausgeben wollen, müssen wir unsere eigene lokale `int`-Variable (z.B. `position`) deklarieren, sodass wir im Rumpf der Schleife in etwa Folgendes schreiben können:

```
System.out.println(position + ": " + dateiname);
```

Versuchen Sie, hierfür eine Version von `alleDateienAusgeben` entsprechend zu erweitern. *Hinweis:* Sie werden eine lokale Variable `position` in der Methode deklarieren müssen, sowie eine Anweisung, die deren Wert innerhalb der `for-each`-Schleife um eins aktualisiert.

Diese Übung veranschaulicht, dass die `for-each`-Schleife nicht wirklich für die Verwendung einer separaten Indexvariablen geeignet ist.

Wir haben nun gesehen, wie wir mit einer `for-each`-Schleife eine bestimmte Operation (den Schleifenrumpf) auf jedes Element einer Sammlung anwenden können. Das ist ein großer Schritt vorwärts, aber er löst noch nicht alle unsere Probleme. Manchmal brauchen wir etwas mehr Kontrolle und Java bietet uns ein weiteres Schleifenkonstrukt an, mit dem wir mehr machen können: die *while-Schleife*.

4.9.2 Eine Sammlung selektiv verarbeiten

Die Methode `alleDateienAusgeben` veranschaulicht, wie nützlich eine `for-each`-Schleife ist: Sie bietet über die Variable, die im Schleifenkopf deklariert ist, nacheinander Zugriff auf jedes Element in einer Sammlung. Sie liefert uns zwar nicht die Indexpositionen der einzelnen Elemente, aber diese werden nicht immer benötigt, sodass das nicht notwendigerweise ein Problem darstellt.

Die Möglichkeit, auf jedes Element in der Sammlung zuzugreifen, bedeutet allerdings nicht, dass wir auf jedem Element das Gleiche ausführen müssen. Wir können selektiver sein. Vielleicht wollen wir nur die Tracks eines bestimmten Interpreten ausgeben oder suchen nach allen Tracks, die ein bestimmtes Wort im Titel enthalten. Es gibt nichts, was uns davon abhalten sollte, da der Rumpf einer `for-each`-Schleife nur ein normaler Block ist, in dem wir jede beliebige Java-Anweisung verwenden können. Deshalb sollte es kein Problem bereiten, eine `if`-Anweisung im Rumpf einzufügen, um die gewünschten Dateien auszuwählen.

Listing 4.4 zeigt eine Methode, die genau die Dateinamen der Sammlung ausgibt, die eine bestimmte Zeichenkette beinhalten.

Listing 4.4

Ausgewählte
Elemente der
Sammlung
ausgeben.

```

/**
 * Liste die Namen aller Dateien auf, die mit einer zu suchenden Zeichenfolge übereinstimmen.
 * @param suchbegriff die Zeichenfolge, die im Namen enthalten sein muss
 */
public void bestimmteDateienAusgeben(String suchbegriff)
{
    for(String dateiname : dateien) {
        if(dateiname.contains(suchbegriff)) {
            // Treffer.
            System.out.println(dateiname);
        }
    }
}

```

Mithilfe einer **if**-Anweisung und dem booleschen Ergebnis der **contains**-Methode der Klasse **String** können wir „filtern“, welche Dateien ausgegeben werden sollen und welche nicht. Wenn der Dateiname nicht mit den Filterkriterien (dem Test in der **if**-Anweisung) übereinstimmt, wird er einfach übergangen – es wird kein **else**-Teil benötigt. Das Filterkriterium können wir frei wählen.

Übung 4.25 Fügen Sie die Methode **bestimmeDateienAusgeben** aus Listing 4.4 Ihrer letzten Projektversion hinzu. (Wenn Sie noch keine eigene Version angelegt haben, können Sie auch *Musiksammlung-v3* verwenden.) Vergewissern Sie sich, dass die Methode nur Dateien ausgibt, die dem Kriterium entsprechen. Versuchen Sie, einen Suchbegriff anzugeben, für den es keine Übereinstimmung in den Dateinamen gibt. Wird in diesem Fall irgendetwas ausgegeben?

Übung 4.26 Zusatzaufgabe. Gibt es in der Methode **bestimmeDateienAusgeben** eine Möglichkeit, nach Beenden der **for-each**-Schleife eine Nachricht auszugeben, die darüber informiert, dass kein Dateiname mit dem Suchbegriff übereinstimmt? *Hinweis:* Verwenden Sie eine lokale boolesche Variable.

Übung 4.27 Schreiben Sie in Ihrer Projektversion eine Methode, die alle Tracks eines bestimmten Interpreten hintereinander anspielt. Die Methode **bestimmeDateienAusgeben** veranschaulicht die Grundstruktur, die Sie für diese Methode benötigen. Achten Sie darauf, einen Interpreten auszuwählen, von dem Sie mehr als einen Track gespeichert haben. Verwenden Sie die **MusikSammlung**-Methode **dateiAnspielen** (zu finden in *Musiksammlung-v3*). Die Methode **dateiAnspielen** spielt den Anfang eines Stücks (ca. 15 Sekunden) und kehrt dann zurück.

Übung 4.28 Schreiben Sie den Kopf einer **for-each**-Schleife, die eine **ArrayList<Track>** namens **tracks** verarbeitet. Machen Sie sich noch keine Gedanken um den Schleifenrumpf.

4.9.3 Probleme bei der Verwendung von Zeichenketten

Aus den obigen Übungen wird ersichtlich, dass allein Dateinamen, die alle Einzelheiten zu den Musikstücken enthalten, nicht wirklich befriedigend sind. Angenommen, Sie wollten nach allen Musikstücken suchen, die im Titel das Wort „Liebe“ aufweisen. Mit der einfachen Suchtechnik, wie wir sie oben beschrieben haben,

finden wir auch Musikstücke, bei denen der Begriff „Liebe“ im Namen des Interpreten (z.B. Liebermann) vorkommt. Das mag auf den ersten Blick kein besonders großes Problem sein, wirkt aber etwas „billig“. Das heißt, es sollte mit nur wenig Mehraufwand möglich sein, eine bessere Version zu erstellen. Was wir dazu benötigen, ist eine separate Klasse, sagen wir **Track**, die die Angaben zu Interpret und Titel unabhängig vom Dateinamen speichert. Dann könnten wir einfacher die Titel separat von den Interpreten abgleichen. Die `ArrayList<String>` würde dann zu einer `ArrayList<Track>`.

Bei der Weiterentwicklung unseres *Musiksammlung*-Projekts in den folgenden Abschnitten werden wir uns letztendlich durch Einführen einer **Track**-Klasse einer besseren Struktur annähern.

4.9.4 Zusammenfassung der for-each-Schleife

Die **for-each**-Schleife wird immer verwendet, um über eine Sammlung zu iterieren. Sie bietet die Möglichkeit, nacheinander jedes Element in der Sammlung anzusprechen und auf beliebige Art und Weise zu verarbeiten. Wir können mit jedem Element das Gleiche machen (wie bei der Ausgabe der ganzen Liste) oder die Liste filtern und Elemente selektieren (wie bei der Ausgabe einer Teilmenge der Sammlung). Der Rumpf der Schleife kann beliebig kompliziert sein.

Die Einfachheit dieses Konzepts weist gezwungenermaßen einige Beschränkungen auf. Eine Beschränkung ist zum Beispiel, dass wir die Sammlung während der Iteration weder durch Hinzufügen neuer Elemente erweitern noch durch Entfernen von Elementen verkleinern können. Das heißt jedoch nicht, dass wir die Zustände der Objekte, die bereits in der Sammlung sind, nicht ändern können.

Wir haben auch gesehen, dass die **for-each**-Schleife uns keine Indexwerte für die Elemente in der Sammlung liefert. Wenn wir dies wünschen, müssen wir unsere eigene lokale Variable deklarieren und verwalten. Der Grund dafür ist wie immer die Abstraktion. In Zusammenhang mit Sammlungen und der Iteration darüber sollten Sie sich zwei Dinge merken:

- Eine **for-each**-Schleife ist eine allgemeine Kontrollstruktur, mit der Sie über verschiedene Arten von Sammlungen iterieren können.
- Es gibt einige Arten von Sammlungen, bei denen die darin gespeicherten Elemente nicht automatisch mit ganzen Zahlen als Indexwerte verbunden werden. Einige davon werden Sie in Kapitel 6 kennenlernen.

Die **for-each**-Schleife abstrahiert sozusagen die Aufgabe, eine vollständige Sammlung Element für Element zu verarbeiten, und zwar für verschiedene Arten von Sammlungen. Die Einzelheiten darüber müssen wir nicht wissen.

Eine der Fragen, die wir noch nicht gestellt haben, ist, ob eine **for-each**-Schleife verwendet werden kann, wenn wir mitten in der der Sammlung mit der Verarbeitung aufhören. Angenommen wir spielen nicht jedes Musikstück des von uns gewählten Interpreten, sondern nur das erste und gehen nicht weiter. Im Prinzip *ist* das mit einer **for-each**-Schleife möglich. Aber Erfahrung hat uns gelehrt, dass es nicht ratsam ist, eine **for-each**-Schleife für Aufgaben zu verwenden, in der nicht die ganze Sammlung verarbeitet werden muss. Mit anderen Worten, wir empfehlen die Ver-

wendung einer **for-each**-Schleife nur, wenn Sie *definitiv die ganze Sammlung* verarbeiten wollen. Das bedeutet, sobald Sie die Schleife gestartet haben, wissen Sie genau, wie oft der Rumpf ausgeführt wird – und zwar entsprechend der Größe der Sammlung. Dieser Stil wird oft als *bestimmte Iteration (definite iteration)* bezeichnet. Bei Aufgaben, die mittendrin abgebrochen werden sollen, gibt es geeignetere Schleifen – zum Beispiel die *while-Schleife*, die wir im Folgenden vorstellen. In diesen Fällen ist die Anzahl, wie oft der Schleifenrumpf ausgeführt werden soll, nicht so sicher; sie hängt oft davon ab, was während der Iteration passiert. Dieser Stil wird oft als *unbestimmte Iteration (indefinite iteration)* genannt und ist Thema des nächsten Abschnitts.

4.10 Unbestimmte Iteration

Mit der **for-each**-Schleife konnten wir erste Erfahrungen mit dem Prinzip der wiederholten Ausführung von Aktionen sammeln. Die Anweisungen im Schleifenrumpf werden nacheinander auf jedes Element der Sammlung ausgeführt. Der Durchlauf ist beendet, wenn wir das Ende der Sammlung erreicht haben. Eine **for-each**-Schleife bietet *bestimmte Iteration*, das heißt, wenn der Zustand einer bestimmten Sammlung gegeben ist, wird der Schleifenrumpf genau so oft ausgeführt, wie Elemente in der Sammlung gespeichert sind. Es gibt jedoch viele Situationen, in denen wir wiederholt einige Aktionen ausführen wollen, ohne im Voraus genau zu wissen, wie viele Durchläufe es geben wird. In diesen Fällen ist eine **for-each**-Schleife keine große Hilfe.

Angenommen, Sie haben Ihre Schlüssel verloren und müssen sie finden, bevor Sie das Haus verlassen können. Eine solche Suche ist ein Beispiel für eine unbestimmte Iteration, da es viele Orte gibt, wo sich der Schlüssel befinden könnte, aber Sie nicht im Voraus sagen können, wie viele Orte Sie durchsuchen müssen, bevor Sie die Schlüssel finden. Wenn Sie das wüssten, könnten Sie ja auch gleich zum vermeintlichen Fundort gehen! Stattdessen werden Sie wahrscheinlich im Kopf eine Liste der möglichen Fundorte erstellen und diese nacheinander absuchen, bis Sie die Schlüssel haben. Nachdem Sie sie gefunden haben, werden Sie sicherlich die Suche abbrechen wollen, anstatt die Liste weiter durchzugehen (was sinnlos wäre).

Bei diesem Beispiel haben wir es mit einer *unbestimmten Iteration* zu tun: die (Such-)Aktion wird beliebig oft ausgeführt, bis die Aufgabe gelöst ist. Szenarien wie die Schlüsselsuche gibt es in der Programmierung häufig. Wir werden zwar nicht immer nach etwas suchen, aber es gibt oft Situationen, in denen wir etwas immer wieder tun wollen, bis die Wiederholung nicht länger benötigt wird. Genau genommen treten sie so häufig auf, dass die meisten Programmiersprachen dafür zumindest ein, wenn nicht sogar mehrerer Schleifenkonstrukte zur Verfügung stellen. Da das, was wir mit diesen Schleifenkonstrukten erreichen wollen, normalerweise komplizierter ist, als von Anfang bis Ende über eine ganze Sammlung zu iterieren, sind sie schwerer zu verstehen. Der Aufwand lohnt sich jedoch, da Sie damit wesentlich mehr machen können. Wir legen den Schwerpunkt hier auf Javas *while-Schleife*, weil sie sehr starke Ähnlichkeit mit Schleifen in anderen Programmiersprachen aufweist.

4.10.1 Die while-Schleife

Eine **while**-Schleife besteht aus einem Schleifenkopf und einem Schleifenrumpf, von denen der Rumpf mehrfach ausgeführt werden kann. Im Folgenden sehen Sie die Struktur einer **while**-Schleife, bei der die boolesche Bedingung und der Schleifenrumpf Pseudocode sind, der Rest hingegen Java-Syntax:

```
while(boolesche Bedingung) {
    Schleifenrumpf
}
```

Wir erkennen, dass eine **while**-Schleife durch das Schlüsselwort **while** eingeleitet wird, gefolgt von einer booleschen Bedingung. Diese Bedingung steuert letztendlich, wie oft eine Schleife ausgeführt wird. Die Bedingung wird ausgewertet, wenn die Programmsteuerung das erste Mal in die Schleife eintritt; nach jeder Ausführung des Schleifenrumpfes wird sie erneut ausgewertet. Dieser wiederholten Auswertung verdankt eine **while**-Schleife ihren unbestimmten Charakter. Wenn die Auswertung der Bedingung **true** liefert, dann wird der Rumpf ausgeführt; sobald die Auswertung **false** liefert, ist die Iteration beendet. In diesem Fall wird der Schleifenrumpf übersprungen und die Ausführung mit dem fortgesetzt, was nach dem Rumpf im Quelltext folgt. Beachten Sie, dass die Bedingung bereits beim ersten Mal zu **false** ausgewertet werden kann, sodass der Rumpf überhaupt nicht ausgeführt wird. Dies ist ein wichtiges Merkmal der **while**-Schleife: Der Rumpf kann auch gar nicht ausgeführt werden, anstatt immer zumindest einmal.

Bevor wir ein richtiges Java-Beispiel betrachten, wollen wir unsere oben beschriebene Schlüsseljagd in Pseudocode ausdrücken, um ein Gefühl dafür zu bekommen, wie eine **while**-Schleife funktioniert. Eine Möglichkeit, die Suche auszudrücken, lautet:

```
while(die Schlüssel fehlen) {
    suche am nächsten Ort
}
```

Wenn wir die Schleife das erste Mal erreichen, wird die Bedingung ausgewertet und die Schlüssel fehlen. Das bedeutet, wir treten in den Rumpf der Schleife ein und suchen am nächsten Ort auf unserer Liste. Nachdem wir das gemacht haben, kehren wir zurück zu der Bedingung und werten sie erneut aus. Haben wir die Schlüssel gefunden, ist die Schleife beendet und wir können den Schleifenrumpf überspringen und das Haus verlassen. Sind die Schlüssel hingegen immer noch unauffindbar, treten wir erneut in den Schleifenrumpf ein und suchen am nächsten Ort. Dieser Prozess wiederholt sich so lange, bis die Schlüssel gefunden sind.¹

Beachten Sie, dass wir genauso gut die Schleifenbedingung auch anders hätten ausdrücken können:

```
while(nicht (die Schlüssel gefunden)) {
    suche am nächsten Ort
}
```

¹ An dieser Stelle wollen wir die Möglichkeit außer Acht lassen, dass die Schlüssel nicht gefunden werden. Allerdings wird die Berücksichtigung dieser Möglichkeit sehr wichtig, wenn wir konkrete Java-Beispiele betrachten.

Die Unterschiede sind nur geringfügig. In dem einen Pseudocode wird der Zustand als etwas ausgedrückt, das zu ändern ist, im anderen als Ziel, das noch nicht erreicht wurde. Nehmen Sie sich die Zeit und studieren Sie beide Versionen sorgfältig, um sicherzugehen, dass Sie alles verstanden haben. Beide sind gültig und spiegeln Entscheidungen wider, die wir für unsere Ausdrücke treffen müssen, wenn wir später konkrete Schleifen schreiben. In beiden Fällen führt das, was wir in den Schleifenrumpf setzen, beim Finden der Schlüssel dazu, dass die Schleifenbedingungen bei der nächsten Auswertung von **true** auf **false** umspringen.

Übung 4.29 Angenommen, wir drücken die erste Version der Schlüssel-suche in Pseudocode wie folgt aus:

```
boolean fehlt = true;
while(fehlt) {
    if(die Schlüssel sind am nächsten Ort) {
        fehlt = false;
    }
}
```

Versuchen Sie die zweite Version auszudrücken, indem Sie folgendes Gerüst vervollständigen:

```
boolean gefunden = false;
while(...) {
    if(die Schlüssel sind am nächsten Ort) {
        ...
    }
}
```

4.10.2 Mithilfe einer Indexvariablen iterieren

Als erste **while**-Schleife in korrektem Java schreiben wir eine Version der Methode **alleDateienAusgeben** aus Listing 4.3. Sie veranschaulicht zwar nicht den unbestimmten Charakter von **while**-Schleifen, liefert aber einen nützlichen Vergleich mit der gleichwertigen, schon bekannten **for-each**-Schleife. Die Version der **while**-Schleife finden Sie in Listing 4.5. Das wichtigste Merkmal ist die **int**-Variable **index**, die sowohl dazu verwendet wird, auf die Elemente der Liste zuzugreifen, als auch um die Häufigkeit der Iterationen zu kontrollieren.

Listing 4.5

Benutzung einer **while**-Schleife zur Ausgabe der Tracks.

```
/**
 * Gib eine Liste aller Dateien in der Sammlung aus.
 */
public void alleDateienAusgeben()
{
    int index = 0;
    while(index < dateien.size()) {
        String dateiname = dateien.get(index);
        System.out.println(dateiname);
        index++;
    }
}
```

Es ist direkt ersichtlich, dass diese Version der **while**-Schleife programmieretechnisch etwas aufwendiger ist:

- Wir müssen eine Variable für den Listenindex deklarieren und wir müssen sie selbst für das erste Listenelement mit 0 initialisieren. Die Variable muss außerhalb der Schleife deklariert werden.
- Wir müssen herausfinden, wie wir die Schleifenbedingung so ausdrücken, dass die Schleife sicher rechtzeitig beendet wird.
- Die Listenelemente werden nicht automatisch für uns aus der Sammlung geholt und einer Variablen zugewiesen. Stattdessen müssen wir das selbst tun, mithilfe der Methode **get** der Klasse **ArrayList**. Die Variable **dateiname** ist lokal zum Schleifenrumpf.
- Wir müssen daran denken, dass wir die Indexvariable (**index**) selbst erhöhen, damit die Schleifenbedingung irgendwann **false** wird, wenn wir das Ende der Liste erreicht haben.

Die letzte Anweisung im Rumpf der **while**-Schleife zeigt einen speziellen Operator zum Erhöhen einer numerischen Variablen um den Wert 1:

```
index++
```

Dies ist äquivalent zu

```
index = index + 1
```

Bisher war die **for-each**-Schleife klar besser für unsere Zwecke geeignet. Sie ist leichter zu schreiben und sie ist sicherer. Sie ist deshalb sicherer, weil garantiert ist, dass sie immer zu einem Ende kommt. In der Version mit der **while**-Schleife ist es möglich, einen Fehler zu machen, der zu einer Endlosschleife führt. Wenn wir das Erhöhen der Indexvariablen vergessen hätten (die letzte Zeile im Schleifenrumpf), dann würde die Schleifenbedingung niemals **false** werden und die Schleife würde endlos weiterlaufen. Das ist ein typischer Programmierfehler, der auch erfahrenen Programmierern immer einmal wieder unterläuft. Ein solches Programm wird immer wieder von vorn ausgeführt. Wenn die Schleife in einer solchen Situation keine Ausgabeanweisung enthält, dann scheint das Programm zu hängen: Es scheint nichts zu tun und reagiert nicht auf Mausklicks oder Tastendrücke. Tatsächlich macht das Programm aber sehr viel: Es führt wieder und wieder die Schleife aus, aber wir können keine Auswirkungen sehen und das Programm scheint gestorben zu sein. In BlueJ lässt sich dies oft daran erkennen, dass der rot-weiß gestreifte Laufstatus-Anzeiger aktiv bleibt, während das Programm anscheinend nichts macht.

Worin liegen also die Vorteile einer **while**-Schleife gegenüber einer **for-each**-Schleife? Es gibt zwei: Erstens muss eine **while**-Schleife nicht mit einer Sammlung verbunden sein – wir können Schleifen auf Basis jeder nur denkbaren Bedingung formulieren, die sich als boolescher Ausdruck schreiben lässt. Zweitens müssen wir, selbst wenn wir mit der Schleife eine Sammlung verarbeiten, nicht jedes Element verarbeiten. Wir können die Schleife bei Bedarf früher beenden, indem wir eine weitere Komponente in die Schleifenbedingung aufnehmen, die ausdrückt, warum wir abbrechen wollen. Genau genommen drückt natürlich die Schleifenbedingung aus, ob wir fortfahren wollen, und die Negation davon ist der Grund, warum die Schleife stoppt.

Der Vorteil einer expliziten Indexvariablen ist, dass wir deren Wert innerhalb und außerhalb der Schleife verwenden können – eine Tatsache, auf die wir bei **for-each**-Schleifen verzichten müssen. Wir können also den Index mit ausgeben, wenn wir wollen. Das erleichtert uns die Auswahl eines Musikstücks anhand seiner Position in der Ausgabe. Zum Beispiel:

```
int index = 0;
while(index < dateien.size()) {
    String dateiname = dateien.get(index);
    // Vor dem Dateinamen den Track-Index ausgeben
    System.out.println(index + ": " + dateiname);
    index++;
}
```

Eine lokale Indexvariable zu haben, ist insbesondere für das Durchsuchen von Liste hilfreich, da sie speichert, wo das Element sich befand – und auch dann noch verfügbar ist, wenn die Schleife beendet ist. Doch dazu im nächsten Abschnitt mehr.

4.10.3 Eine Sammlung durchsuchen

Die Suche ist eine der wichtigsten Formen der Iteration, mit der Sie es zu tun bekommen werden. Deshalb ist es wichtig, die wesentlichen Elemente gut zu kennen. Die Art von Schleifenstruktur tritt beim Programmieren in der Praxis immer wieder auf.

Das Hauptmerkmal einer Suche ist, dass sie mit einer *unbestimmten Iteration* verbunden ist, was nicht verwunderlich ist, denn wenn wir genau wüssten, wo wir suchen sollten, müssten wir gar nicht suchen! Stattdessen müssen wir eine Suche einleiten, deren Erfolg von einer unbestimmten Anzahl von Iterationen abhängt. Das bedeutet, dass eine **for-each**-Schleife sich nicht für das Suchen eignet, da sie immer alle Iterationen durchläuft.²

In konkreten Suchsituationen müssen wir berücksichtigen, dass die Suche fehlschlagen kann: Es könnten uns die Suchorte ausgehen. Das heißt, wir müssen normalerweise zwei Szenarien für das Beenden der Schleife ins Auge fassen, wenn wir eine Suchschleife schreiben:

- Die Suche ist nach einer unbestimmten Anzahl von Iterationen erfolgreich.
- Die Suche schlägt fehl, nachdem alle Möglichkeiten ausprobiert wurden.

Beide Szenarien müssen beim Schreiben der Schleifenbedingung berücksichtigt werden. Genauso wie die Schleifenbedingung zu **true** ausgewertet werden sollte, wenn wir ein weiteres Mal über die Schleife iterieren wollen, sollte jedes der Beenden-Kriterien für sich dazu führen, dass die Bedingung zu **false** ausgewertet wird, um die Schleife zu stoppen. Die Tatsache, dass wir das Durchsuchen der ganzen Liste beenden, wenn die Suche fehlschlägt, macht eine fehlgeschlagene Suche nicht zu einem Beispiel der bestimmten Iteration. Das Hauptmerkmal einer bestimmten Iteration ist, dass Sie die Anzahl der Iterationen zu *Beginn der Schleife* festlegen können. Dies ist bei einer Suche nicht der Fall.

² Auch wenn es leider recht häufig genutzte Möglichkeiten gibt, dieses Merkmal einer **for-each**-Schleife zu unterlaufen, betrachten wir sie hier als schlechten Stil und benutzen sie in unseren Beispielen nicht.

Wenn wir eine Indexvariable verwenden, um nacheinander die Elemente einer Sammlung durchzugehen, ist eine fehlgeschlagene Suche leicht zu identifizieren: Die Indexvariable ist über das letzte Element in der Liste hinaus inkrementiert worden. Das ist genau die Situation, die in der Methode `alleDateienAusgeben` in Listing 4.5 durch folgende Bedingung abgedeckt wird:

```
while(index < dateien.size())
```

Die Bedingung drückt aus, dass wir fortfahren wollen, solange der Index im gültigen Bereich der Sammlung liegt; sobald der Index über den Indexbereich hinaus inkrementiert wurde, soll die Schleife aufhören. Diese Bedingung funktioniert auch, wenn die Liste komplett leer ist. In diesem Fall wurde `index` mit 0 initialisiert und der Aufruf der Methode `size` liefert ebenfalls 0 zurück. Da 0 nicht kleiner 0 ist, wird die Schleife nicht ausgeführt, was genau das ist, was wir wollen.

Darüber hinaus müssen wir zu der Bedingung einen zweiten Teil hinzufügen. Seine Aufgabe ist es anzuzeigen, ob wir das gesuchte Element schon gefunden haben, und in diesem Fall die Suche zu beenden. Wir haben in Abschnitt 4.10.1 und **Übung 4.29** gesehen, dass wir dies über entsprechend gesetzte boolesche Variablen oft positiv oder negativ ausdrücken können:

- Eine Variable `amSuchen` (oder `fehlt`), die am Anfang auf `true` gesetzt ist, könnte die Suche so lange weiterführen, bis sie in der Schleife auf `false` gesetzt wird, weil das Element gefunden wurde.
- Eine Variable `gefunden`, die am Anfang auf `false` gesetzt ist und in der Bedingung als `!gefunden` verwendet wird, könnte die Suche so lange weiterführen, bis sie auf `true` gesetzt wird, weil das Element gefunden wurde.

Im Folgenden finden Sie zwei entsprechende Quelltextabschnitte, die die komplette Bedingung für beide Fälle ausdrücken:

```
int index = 0;
boolean amSuchen = true;
while(index < dateien.size() && amSuchen)
```

oder

```
int index = 0;
boolean gefunden = false;
while(index < dateien.size() && !gefunden)
```

Studieren Sie die Quelltextabschnitte sorgfältig. Beide bieten die gleiche Art der Schleifensteuerung, werden aber leicht unterschiedlich ausgedrückt. Zur Erinnerung: Die Bedingung muss als Ganzes zu `true` ausgewertet werden, wenn wir weitersuchen wollen, und zu `false`, wenn wir aus einem beliebigen Grund aufhören wollen. Wir haben den Und-Operator (`&&`) bereits in Kapitel 3 angesprochen. Er wird nur zu `true` ausgewertet, wenn *beide* seiner Operanden `true` sind.

Die volle Version einer Methode, die den ersten Dateinamen sucht, der einem gegebenen Suchbegriff entspricht, ist in Listing 4.6 (*Musiksammlung-v4*) zu finden. Die Methode liefert den Index des Elements als Ergebnis zurück. Beachten Sie, dass wir eine Möglichkeit finden müssen, den Aufrufer der Methode darüber zu informieren, falls die Suche fehlgeschlagen ist. Wir haben uns in diesem Fall dafür entschieden, einen Wert zurückzugeben, der eindeutig keine gültige Posi-

tion in der Sammlung ist, da er negativ ist. Diese Vorgehensweise ist typisch für Suchsituationen: Um einen Fehlschlag anzuzeigen, geben wir einen Wert zurück, der *außerhalb des Gültigkeitsbereichs* liegt.

Listing 4.6

Das erste übereinstimmende Element in einer Liste suchen.

```
/**
 * Finde den Index der ersten Datei, die mit dem Suchbegriff übereinstimmt.
 * @param suchbegriff die Zeichenfolge, die im Namen enthalten sein muss.
 * @return der Index der ersten Übereinstimmung oder -1, wenn
 *         kein übereinstimmender Dateiname gefunden wurde
 */
public int findeErste(String suchbegriff)
{
    int index = 0;
    // Wir suchen so lange, bis eine Übereinstimmung gefunden wurde.
    boolean amSuchen = true;

    while(amSuchen && index < dateien.size()) {
        String dateiname = dateien.get(index);
        if(dateiname.contains(suchbegriff)) {
            // Eine Übereinstimmung. Wir können die Suche beenden.
            amSuchen = false;
        }
        else {
            // weiter
            index++;
        }
    }

    if(amSuchen) {
        // Nichts gefunden.
        return -1;
    }
    else {
        // Teile mit, wo die Übereinstimmung gefunden wurde.
        return index;
    }
}
```

Es mag verlockend sein zu versuchen, nur eine Bedingung in die Schleife aufzunehmen, auch wenn es zwei verschiedenen Gründe gibt, die Suche zu beenden. Eine Möglichkeit hierfür wäre, dafür zu sorgen, dass der Wert des Index zu groß wird, wenn wir das Gesuchte gefunden haben. Von dieser Praxis wollen wir aber lieber abraten, da der Code dann die Beenden-Kriterien der Schleife nicht direkt widerspiegelt und Klarheit immer unser oberstes Ziel ist.

4.10.4 Einige Beispiele ohne Sammlung

Schleifen werden nicht nur bei Sammlungen eingesetzt. Es gibt viele Situationen, in denen wir Anweisungsblöcke wiederholen wollen, die nichts mit einer Sammlung zu tun haben. Im Folgenden sehen Sie ein Beispiel, der alle geraden Zahlen von 0 bis 30 ausgibt:

```
int index = 0;
while(index <= 30) {
    System.out.println(index);
    index = index + 2;
}
```

Dies ist ein Beispiel für die Verwendung einer **while**-Schleife für eine bestimmte Iteration, da von Anfang an klar ist, wie viele Zahlen ausgegeben werden. Wir können in diesem Fall keine **for-each**-Schleife verwenden, da diese nur beim Iterieren

über Sammlungen zum Einsatz kommen. Später werden wir noch einer dritten, verwandten Schleife (der *for-Schleife*) begegnen, die sich noch besser für dieses bestimmte Beispiel eignet. Um zu überprüfen, ob Sie **while**-Schleifen verstanden haben, die nicht bei Sammlungen eingesetzt werden, sollten Sie die folgenden Übungen bearbeiten.

Übung 4.30 Schreiben Sie eine **while**-Schleife (beispielsweise in einer Methode **vielfacheVonFuenf**), die alle Vielfachen von 5 zwischen 10 und 95 ausgibt.

Übung 4.31 Schreiben Sie eine **while**-Schleife, die die Werte von 1 bis 10 addiert und die Summe nach Beenden der Schleife ausgibt.

Übung 4.32 Schreiben Sie eine Methode **summieren** mit einer **while**-Schleife, die alle Zahlen zwischen zwei Zahlen **a** und **b** aufsummiert. Die Werte von **a** und **b** können der Methode als Parameter übergeben werden.

Übung 4.33 *Zusatzaufgabe.* Schreiben Sie eine Methode **istPrim(int n)**, die **true** liefert, wenn der Parameter **n** eine Primzahl ist, und **false**, wenn sie keine ist. In der Implementierung der Methode können Sie eine **while**-Schleife schreiben, die **n** durch alle Zahlen zwischen 2 und (**n-1**) teilt und überprüft, ob die Division eine ganze Zahl liefert. Sie können für diesen Test den Modulo-Operator (%) verwenden, indem Sie mit ihm überprüfen, ob die **int**-Division zu einem Rest von null führt (siehe die Beschreibung des Modulo-Operators in Abschnitt 3.8.3).

Übung 4.34 In der Methode **findeErste** fragt die Bedingung in der Schleife die Sammlung **dateien** wiederholt, wie viele Dateien sie speichert. Ändert sich der von **size** gelieferte Wert von einer Prüfung zur nächsten? Lautet Ihre Antwort nein, dann schreiben Sie die Methode so um, dass die Anzahl der Dateien nur einmal festgelegt und in einer lokalen Variablen gespeichert wird, bevor die Schleife ausgeführt wird. Verwenden Sie dann diese Variable und nicht den Aufruf von **size** in der Schleifenbedingung. Prüfen Sie, dass diese Version die gleichen Ergebnisse liefert. Wenn Sie Schwierigkeiten haben, diese Übung zu beenden, setzen Sie den Debugger ein, um zu sehen, wo Sie Fehler gemacht haben.

4.11 Verbesserung der Struktur – die Klasse Track

Wir haben bereits mehrmals erlebt, dass die Verwendung von Zeichenketten zur Speicherung der ganzen Details der Musikstücke nicht gänzlich zufriedenstellend ist und unseren Musikplayer relativ billig erscheinen lässt. Jeder handelsübliche Player würde erlauben, Musikstücke nach Interpret, Titel, Album, Genre usw. zu suchen, und außerdem weitere Funktionen bieten, wie z.B. die Anzeige von Dauer und Nummer der Musikstücke. Eine der Stärken der Objektorientierung ist, dass sie uns erlaubt, Klassen zu entwerfen, die die inhärente Struktur und die Verhaltensweisen der realen Entitäten modellieren, die wir versuchen darzustellen. Dies erreichen wir,

indem wir Klassen schreiben, deren Datenfelder und Methoden denen der Attribute entsprechen. Inzwischen wissen wir genug über das Schreiben von einfachen Klassen mit Datenfeldern, Konstruktoren sowie sondierenden und verändernden Methoden, dass wir leicht eine Klasse **Track** für Musikstücke entwerfen können, die Datenfelder aufweist, um zum Beispiel getrennt Informationen über den Interpreten und den Titel zu speichern. Auf diese Weise können wir mit den Objekten in der Musiksammlung interagieren, wie es eher unserer natürlichen Vorgehensweise entspricht.

Höchste Zeit also, mit dem Speichern der Trackinformationen als Zeichenketten aufzuhören, da eine getrennte Klasse **Track** ein viel geeigneterer Weg ist, die wichtigsten Datenelemente (Tracks), die wir in dem Programm verwenden, darzustellen. Wir sind jedoch nicht allzu ehrgeizig. Eine der größeren Hürden, die wir überwinden müssen, ist, wie wir an die verschiedenen Informationen gelangen, die wir in jedem **Track**-Objekt speichern wollen. Eine Möglichkeit wäre, den Benutzer aufzufordern, beim Hinzufügen jeder Musikdatei zu der Sammlung Interpret, Titel, Genre usw. anzugeben. Das wäre jedoch ziemlich langsam und arbeitsaufwendig. Deshalb haben wir für dieses Projekt einen Satz von Musikdateien gewählt, deren Dateinamen bereits Name des Interpreten und des Titels enthalten. Zusätzlich haben wir eine Hilfsklasse (namens **TrackReader**) für unsere Anwendung geschrieben, die nach allen Musikdateien in einem bestimmten Ordner Ausschau hält und deren Dateinamen heranzieht, um Teile der entsprechenden **Track**-Objekte zu füllen. Wir kümmern uns im Moment noch nicht um die Einzelheiten der Vorgehensweise. (Weiter hinten im Buch besprechen wir die Techniken und Bibliotheksklassen, die in der Klasse **TrackReader** verwendet werden.) Eine Implementierung dieses Entwurfs finden Sie in *Musiksammlung-v5*.

Auf folgende Punkte sollten Sie in dieser Version achten:

- Die wichtigste neue Änderung erfolgt in der Klasse **MusikSammlung**. Es werden in der **ArrayList** keine **String**-Objekte mehr, sondern **Track**-Objekte gespeichert (Listing 4.7). Dies hat Einfluss auf die meisten der bisher entwickelten Methoden.
- Bei der Ausgabe der Trackinformationen in **alleTracksAusgeben** fordern wir das **Track**-Objekt auf, einen **String** zu liefern, der die Details enthält. Das zeigt, dass wir die Klasse **Track** so ausgelegt haben, dass sie dafür verantwortlich ist, die auszugebenden Details wie Interpret und Titel zur Verfügung zu stellen. Es ist ein Beispiel für den sogenannten *Entwurf nach Zuständigkeiten*, den wir in einem Kapitel weiter hinten ausführlicher behandeln.
- In der Methode **spieleTrack** müssen wir dann den Dateinamen aus dem ausgewählten **Track**-Objekt abfragen, bevor wir ihn an den Player übergeben.
- In der Musikbibliothek haben wir Code hinzugefügt, der automatisch Dateinamen aus dem Ordner *audio* ausliest, sowie einige Ausgabeanweisungen, um die Informationen anzuzeigen.

```

import java.util.ArrayList;

/**
 * Eine Klasse zur Verwaltung von Audiotracks.
 * Die einzelnen Tracks können abgespielt werden.
 *
 * @author David J. Barnes und Michael Kölling
 * @version 2016.02.29
 */
public class MusikSammlung
{
    // Eine ArrayList, in der die Musik-Tracks gespeichert werden können.
    private ArrayList<Track> tracks;
    // Ein Player zum Abspielen der Musik-Tracks.
    private MusikPlayer player;
    // Ein Reader, der Musikdateien lesen und als Tracks laden kann.
    private TrackReader reader;

    /**
     * Erzeuge eine MusikSammlung.
     */
    public MusikSammlung()
    {
        tracks = new ArrayList<>();
        player = new MusikPlayer();
        reader = new TrackReader();
        liesBibliothek("../audio");
        System.out.println("Musikbibliothek wurde geladen. " + gibAnzahlTracks() + " Tracks.");
        System.out.println();
    }

    /**
     * Füge der Sammlung einen Track hinzu.
     * @param track der hinzuzufügende Track
     */
    public void trackHinzufuegen(Track track)
    {
        tracks.add(track);
    }

    /**
     * Spiele einen Track aus der Sammlung.
     * @param index der Index des abzuspielenden Tracks
     */
    public void spieleTrack(int index)
    {
        if(geltigerIndex(index)) {
            Track track = tracks.get(index);
            player.starteAbspielen(track.gibName());
            System.out.println("Sie hoeren gerade: " + track.gibInterpret() + " - " + track.gibTitel());
        }
    }

    /**
     * Gib eine Liste aller Tracks in der Sammlung aus.
     */
    public void alleTracksAusgeben()
    {
        System.out.println("Track-Liste: ");

        for(Track track : tracks) {
            System.out.println(track.gibDetails());
        }
        System.out.println();
    }

    // weitere Methoden ausgelassen
}

```

Listing 4.7

Die Verwendung von Track in der Klasse MusikSammlung.

Auch wenn ersichtlich ist, dass die Einführung einer Klasse **Track** einige der alten Methoden etwas verkompliziert hat, führt die Arbeit mit speziellen **Track**-Objekten letztendlich zu einer viel besseren Struktur des ganzen Programms und erlaubt uns, die Klasse **Track** beliebig auszubauen, sodass sie mehr als nur Namen von Audio-dateien repräsentiert. Die Information ist nicht nur besser strukturiert: die Verwendung einer **Track**-Klasse ermöglicht es uns, umfassendere Informationen zu speichern, wenn wir es möchten, wie zum Beispiel das Bild vom Cover des Albums.

Dank der besseren Strukturierung der Trackinformationen können wir jetzt viel besser nach Tracks suchen, die bestimmten Kriterien entsprechen. Wenn wir zum Beispiel alle Tracks suchen, die das Wort „Liebe“ im Titel haben, können wir dazu folgenden Code verwenden, ohne Gefahr zu laufen, unerwünschte Treffer aufgrund Übereinstimmungen mit den Interpretennamen zu erzielen:

```
/**
 * Liste alle Tracks, die einen gegebenen Suchbegriff im Titel enthalten.
 * @param suchbegriff der zu suchende Begriff.
 */
public void sucheInTitel(String suchbegriff)
{
    for(Track track : tracks) {
        String titel = track.gibTitel();
        if(titel.contains(suchbegriff)) {
            System.out.println(track.gibDetails());
        }
    }
}
```

Übung 4.35 Fügen Sie zu der Klasse **Track** ein Datenfeld **abspielFrequenz** hinzu. Ergänzen Sie Methoden, die die Zählung auf null setzen und um eins inkrementieren.

Übung 4.36 Lassen Sie die Klasse **MusikSammlung** die Abspielfrequenz eines Tracks bei jedem Abspielen inkrementieren.

Übung 4.37 Fügen Sie ein weiteres Datenfeld Ihrer Wahl zu der Klasse **Track** hinzu sowie sondierende und verändernde Methoden, um es abzufragen und zu manipulieren. Suchen Sie nach einer Möglichkeit, diese Information in Ihrer Version des Projekts zu verwenden. Bauen Sie die Information beispielsweise in die Ausgabe ein oder richten Sie in der Klasse **MusikSammlung** eine Methode zum Setzen der Information ein.

Übung 4.38 Wenn Sie zwei Tracks abspielen, ohne den ersten zu stoppen, werden beide gleichzeitig abgespielt. Das ist nicht besonders nützlich. Ändern Sie Ihr Programm so, dass das Abspielen eines Tracks automatisch angehalten wird, wenn ein anderer Track gestartet wurde.

4.12 Der Typ Iterator

Die Iteration ist ein wichtiges Werkzeug in fast jedem Programmierprojekt, sodass es nicht Wunder nimmt, dass Programmiersprachen normalerweise eine ganze Reihe von Funktionen bereitstellen, mit denen sie die Iteration unterstützen. Jede dieser Funktionen verfügt über jeweils eigene individuelle Merkmale für verschiedene Situationen.

Wir wollen hier für die Iteration über eine Sammlung eine dritte Variante vorstellen, die irgendwo zwischen der **while**-Schleife und der **for-each**-Schleife angesiedelt ist. Sie besteht aus einer **while**-Schleife für die Iteration und einem *Iterator-Objekt* anstelle einer **int**-Indexvariablen, um die Position in der Liste zu speichern. Wir müssen an dieser Stelle große Sorgfalt bei der Benennung walten lassen, da **Iterator** (man beachte das großgeschriebene **I**) eine Java-Klasse ist, wir es aber auch mit einer *Methode* namens **iterator** (kleingeschriebenes **i**) zu tun haben. Achten Sie also sorgfältig auf diese Unterschiede, wenn Sie diesen Abschnitt lesen und später eigenen Code schreiben.

Die Prüfung eines jeden Elements in einer Sammlung kommt so häufig vor, dass wir bereits eine spezielle Kontrollstruktur – die **for-each**-Schleife – kennengelernt haben, die genau für diesen Zweck entwickelt wurde. Zusätzlich bieten Javas Sammlungsklassen einen speziellen, häufig verwendeten Typ, der ebenfalls Iteration unterstützt; **ArrayList** ist hierfür ein typischer Vertreter.

Die Methode **iterator** der Klasse **ArrayList** liefert ein **Iterator**-Objekt. **Iterator** ist ebenfalls im Paket **java.util** deklariert, entsprechend müssen wir eine weitere **import**-Anweisung in den Quelltext einfügen, um den Typ benutzen zu können:

```
import java.util.ArrayList;
import java.util.Iterator;
```

Ein **Iterator** bietet nur vier Methoden, von denen zwei verwendet werden, um über eine Sammlung zu iterieren: **hasNext** und **next**. Keine von ihnen nimmt einen Parameter entgegen, aber beide haben einen Ergebnistyp, der nicht **void** ist, sodass sie in Ausdrücken verwendet werden. Die übliche Art, einen **Iterator** zu benutzen, kann mit folgendem Pseudocode veranschaulicht werden:

```
Iterator<Elementtyp> it = meineSammlung.iterator();
while (it.hasNext()) { solange es noch ein weiteres Element gibt
    mit it.next() das nächste Element abholen
    etwas mit diesem Element tun
}
```

In diesem Fragment benutzen wir zuerst die Methode **iterator** der Klasse **ArrayList**, um ein **Iterator**-Objekt zu bekommen. Beachten Sie, dass **Iterator** ebenfalls ein generischer Typ ist, sodass wir es mit dem Typ der Elemente in der Sammlung parametrisieren, über die wir iterieren. Anschließend benutzen wir diesen **Iterator**, um zu prüfen, ob es noch weitere Elemente gibt (**it.hasNext()**) und um dieses Element dann auch zu bekommen (**it.next()**). Ein wichtiger Punkt an dieser Stelle ist, dass wir jeweils den **Iterator** nach dem nächsten Element fragen, nicht das Sammlungsobjekt. Genau genommen versuchen wir überhaupt nicht im Rumpf der Schleife direkt auf die Sammlung Bezug zu nehmen. Alle Interaktionen mit der Sammlung erfolgen über den **Iterator**.

Konzept

Ein **Iterator** ist ein Objekt, mit dessen Hilfe über alle Elemente einer Sammlung iteriert werden kann (d.h. alle Elemente durchlaufen werden können).

Mithilfe eines **Iterator** können wir eine Methode zur Ausgabe aller Tracks wie in Listing 4.8 schreiben. Der **Iterator** beginnt am Anfang der Sammlung und arbeitet sich, Objekt für Objekt, jedes Mal weiter, wenn wir die Methode **next** aufrufen.

Listing 4.8

Benutzung eines
Iterators zur Ausgabe
aller Tracks.

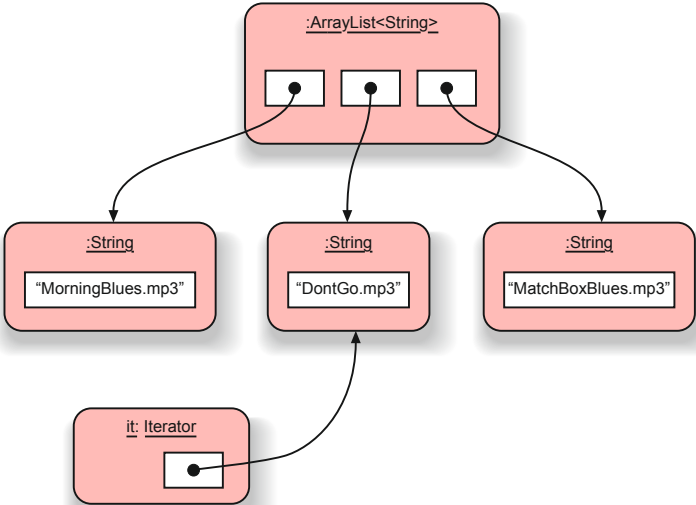
```
/**
 * Gib eine Liste aller Tracks in der Sammlung aus.
 */
public void alleTracksAusgeben()
{
    Iterator<Track> it = tracks.iterator();
    while(it.hasNext()) {
        Track t = it.next();
        System.out.println(t.gibDetails());
    }
}
```

Nehmen Sie sich etwas Zeit, um diese Version der Version gegenüberzustellen, die in Listing 4.7 eine **for-each**-Schleife verwendet, sowie sie mit den beiden Versionen von **alleDateienAusgeben** in Listing 4.3 und Listing 4.4 zu vergleichen. Bemerkenswert an der letzten Version ist, dass wir eine explizite **while**-Schleife verwenden, uns aber nicht mehr um die Indexvariable zu kümmern brauchen. Das liegt daran, dass ein **Iterator** selbst die Information darüber verwaltet, wie weit er bereits eine Sammlung durchlaufen hat. Auf diese Weise ist dem **Iterator** bekannt, ob es noch weitere Elemente gibt (**hasNext**) und welches geliefert werden muss (**next**), wenn es noch ein weiteres gibt.

Um zu verstehen, wie ein **Iterator** funktioniert, müssen Sie sich unter anderem bewusst sein, dass der Aufruf von **next** den **Iterator** veranlasst, das nächste Element in der Sammlung zu liefern und *dann die Position hinter diesem Element einzunehmen*. Deshalb liefern mehrere aufeinanderfolgende Aufrufe von **next** auf einen **Iterator** immer unterschiedliche Elemente. Nachdem **next** aufgerufen worden ist, können Sie nicht mehr zurück zum vorherigen Element. Zum Schluss erreicht der **Iterator** das Ende der Sammlung und liefert dann durch den Aufruf von **hasNext** **false** zurück. Danach würde es zu einem Fehler kommen, wenn versucht würde, **next** auf dieses bestimmte **Iterator**-Objekt aufzurufen – das **Iterator**-Objekt wurde sozusagen „aufgebraucht“ und hat keinen weiteren Nutzen.

Abbildung 4.5

Ein Iterator nach einer
Iteration, der auf das
nächste zu verarbei-
tende Element zeigt.



Auf den ersten Blick scheint **Iterator** gegenüber den vorherigen Möglichkeiten, über eine Sammlung zu iterieren, keine Vorteile zu bieten. Aber die folgenden zwei Abschnitte sollen Ihnen zeigen, warum es so wichtig ist zu wissen, wie man damit arbeitet.

4.12.1 Zugriff mit Index oder über Iteratoren

Wir haben gesehen, dass es mindestens drei Möglichkeiten gibt, um über eine **ArrayList** zu iterieren: Wir können eine **for-each**-Schleife verwenden (wie in Abschnitt 4.9.1), die Methode **get** mit einem Index einsetzen (wie in Abschnitt 4.10.2) oder wir können ein **Iterator**-Objekt verwenden (dieser Abschnitt).

Nach unserem bisherigen Verständnis scheinen alle Ansätze gleich gut zu sein. Der erste Ansatz war möglicherweise etwas leichter zu verstehen, aber am wenigsten flexibel.

Die Benutzung einer **for-each**-Schleife ist die Standardtechnik, wenn alle Elemente einer Sammlung verarbeitet werden sollen (d.h. bestimmte Iteration), denn sie ist für diesen Fall am klarsten. Die beiden letzten Versionen haben den Vorteil, dass die Wiederholungen leichter in der Mitte der Verarbeitung gestoppt werden können (unbestimmte Iteration), sodass sie vorzuziehen sind, wenn nur ein Teil der Sammlung verarbeitet werden muss.

Bei einer **ArrayList** sind die letzten beiden Möglichkeiten mit den **while**-Schleifen tatsächlich gleich gut. Dies ist jedoch nicht immer der Fall. Java bietet neben der **ArrayList** noch etliche weitere Sammlungsklassen an. Wir werden einige davon in den nächsten Kapiteln noch zu sehen bekommen. Für einige dieser Sammlungen ist es entweder unmöglich oder sehr ineffizient, auf einzelne Elemente über einen Index zuzugreifen. Unsere erste Lösung mit einer **while**-Schleife ist deshalb angemessen für eine **ArrayList**, bei einigen anderen Sammlungstypen funktioniert sie aber möglicherweise nicht.

Die letzte Lösung mit einem Iterator hingegen wird von allen Sammlungstypen der Java-Klassenbibliothek unterstützt; sie ist deshalb ein wichtiges *Programmiermuster*, das wir in späteren Projekten noch häufiger verwenden werden.

4.12.2 Elemente entfernen

Bei unserer Entscheidung, welche Schleife am besten geeignet ist, sollten wir auch berücksichtigen, ob wir bei den Schleifendurchläufen Elemente aus der Sammlung entfernen müssen. Stellen wir uns zum Beispiel vor, wir wollten alle Tracks aus unserer Sammlung entfernen, die von einem Interpreten stammen, der uns nicht mehr interessiert.

Den zugehörigen Pseudocode haben wir schnell formuliert:

```
für jeden track in der sammlung {
    wenn track.gibInterpret() nicht mehr gewünschte Interpret ist:
        entferne ihn
}
```

Es stellt sich allerdings heraus, dass dies mit einer **for-each**-Schleife nicht möglich ist. Wenn wir während eines Durchlaufs versuchen, die Sammlung mit einer ihrer

remove-Methoden zu verändern, erhalten wir vom System eine Fehlermeldung über eine unzulässige Veränderung (eine **ConcurrentModificationException**). Dies liegt daran, dass Änderungen an der Sammlung mitten in einem Durchlauf zu recht schwierigen Situationen führen können. Was ist, wenn das Element entfernt wird, an dem wir gerade arbeiten? Wie sollen wir danach das nächste Element finden? Auf diese Fragen gibt es keine befriedigenden allgemeingültigen Antworten. Aus diesem Grunde ist es nicht erlaubt, die **remove**-Methode einer Sammlung während eines Durchlaufs mit einer **for-each**-Schleife zu verwenden.

Eine gute Lösung, um ein Element während eines Durchlaufs zu entfernen, ist, einen **Iterator** zu verwenden, dessen dritte Methode (neben **hasNext** und **next**) **remove** lautet. Diese Methode nimmt keine Parameter entgegen und hat einen Ergebnistyp **void**. Der Aufruf von **remove** entfernt das Element, das vom letzten Aufruf von **next** geliefert wurde. Der zugehörige Code könnte wie folgt aussehen:

```
Iterator<String> it = tracks.iterator();
while (it.hasNext()) {
    Track t = it.next();
    String interpret = t.gibInterpret();
    if (interpret.contains(zuEntfernenderInterpret)) {
        it.remove();
    }
}
```

Erneut sei darauf hingewiesen, dass wir die Sammlungsvariable **tracks** nicht im Schleifenrumpf verwenden. Auch wenn **ArrayList** als auch **Iterator** über **remove**-Methoden verfügen, müssen wir die **remove**-Methode von **Iterator** verwenden und nicht die von **ArrayList**.

Die **remove**-Methode von **Iterator** ist weniger flexibel – wir können keine beliebigen Elemente entfernen. Wir können nur das letzte Element entfernen, das wir von der **Iterator**-Methode **next** erhalten haben. Die Methode **remove** hat jedoch den Vorteil, dass man sie während eines Schleifendurchlaufs verwenden darf. Da der **Iterator** über das Entfernen des Elements informiert ist (und das Entfernen für uns erledigt), kann er dafür sorgen, dass der Durchlauf auch nach dem Entfernen in Übereinstimmung mit dem aktuellen Zustand der Sammlung erfolgt.

Mit einer **for-each**-Schleife sind solche Löschoperationen nicht möglich, da uns darin kein **Iterator** zur Verfügung steht, über den wir löschen könnten. Wir müssen also für solche Fälle auf die Kombination von **while**-Schleife und **Iterator** zurückgreifen.

Technisch gesehen können wir Elemente entfernen, indem wir die **get**-Methode der Sammlung mit einem Schleifenindex aufrufen. In der Praxis ist hiervon allerdings abzuraten, da sich die Indizes der Elemente ändern, wenn man Elemente hinzufügt oder entfernt. Schnell hat man Schleifen programmiert, die nach Manipulationen an der zugrunde liegenden Sammlung mit falschen Indizes arbeiten. Durch die Verwendung eines **Iterator** lassen sich solche Fehler vermeiden.

Übung 4.39 Implementieren Sie in Ihrem Musiksammlung-Beispiel eine Methode, die als Parameter eine Zeichenkette entgegennimmt und dann alle Tracks entfernt, die diese Zeichenkette enthalten.

4.13 Zusammenfassung des Musiksammlung-Beispiels

Am Musiksammlung-Beispiel haben wir gesehen, wie wir ein **ArrayList**-Objekt mithilfe einer Bibliotheksklasse erzeugen und zum Speichern einer beliebigen Anzahl von Objekten in einer Sammlung verwenden können. Wir müssen nicht von vornherein festlegen, wie viele Objekte wir speichern wollen, und das **ArrayList**-Objekt hält automatisch die Information darüber, wie viele Elemente es bereits enthält.

Wir haben diskutiert, wie man mithilfe einer Schleife über alle Elemente einer Sammlung iterieren kann. Java bietet mehrere Schleifenkonstrukte – wir haben hier die *for-each-Schleife* und die *while-Schleife* verwendet. Wir verwenden in der Regel eine **for-each**-Schleife, wenn wir die ganze Sammlung verarbeiten wollen, und die **while**-Schleife, wenn wir nicht im Voraus sagen können, wie viele Durchläufe wir benötigen, oder wenn wir Elemente während der Durchläufe entfernen müssen.

Bei einer **ArrayList** können wir auf die Elemente entweder mit einem Index zugreifen oder wir können mit einem **Iterator**-Objekt alle Elemente durchlaufen. Es lohnt sich, die verschiedenen Umstände zu prüfen, unter denen die verschiedenen Schleifentypen (**for-each** und **while**) eingesetzt werden und warum ein **Iterator** einem ganzzahligen Index vorzuziehen ist, weil Sie Entscheidungen dieser Art immer wieder fällen müssen. Hier die richtige Wahl zu treffen, kann es viel einfacher machen, ein bestimmtes Problem zu lösen.

Übung 4.40 Benutzen Sie das Projekt *Verein* für diese und die folgenden Übungen. Ihre Aufgabe ist es, die Klasse **Verein**, von der bereits ein Grobgerüst vorhanden ist, zu vervollständigen. Die Klasse **Verein** soll **Mitgliedschaft**-Objekte in einer Sammlung mit flexibler Größe speichern.

Definieren Sie in **Verein** ein Datenfeld für eine **ArrayList**. Definieren Sie die notwendige **import**-Anweisung für dieses Datenfeld und überlegen Sie sich genau den Elementtyp der Liste. Erzeugen Sie im Konstruktor das Sammlungsobjekt und weisen Sie es dem Datenfeld zu. Stellen Sie sicher, dass alle Dateien des Projektes übersetzt werden können, bevor Sie mit der nächsten Übung beginnen.

Übung 4.41 Vervollständigen Sie die Methode **anzahlMitglieder** so, dass sie die aktuelle Größe der Sammlung liefert. Bis Sie eine Methode zum Einfügen von Objekten in die Sammlung haben, wird diese natürlich immer null liefern, aber die Methode ist für spätere Tests vorbereitet.

Übung 4.42 Die Mitgliedschaft in einem Verein wird durch eine Instanz der Klasse **Mitgliedschaft** repräsentiert. Eine vollständige Version der Klasse **Mitgliedschaft** ist im Projekt *Verein* bereits vorgegeben und sollte keine Veränderung mehr benötigen. Ein **Mitgliedschaft**-Objekt enthält Informationen über den Namen einer Person sowie den Monat und das Jahr des Zeitpunktes, an dem die Person dem Verein beigetreten ist. Alle Daten zu einer Mitgliedschaft werden angegeben, wenn ein **Mitgliedschaft**-Objekt erzeugt wird. Ein neues **Mitgliedschaft**-Objekt wird in die Sammlung eines **Verein**-Objektes über die Methode **eintreten** eingefügt, die die folgende Signatur hat:

```

/**
 * Füge ein neues Mitglied in die Mitgliederliste ein.
 * @param mitglied Infos über das einzufügende Mitglied.
 */
public void beitreten (Mitgliedschaft mitglied)

```

Vervollständigen Sie die Methode **beitreten**.

Wenn Sie ein neues **Mitgliedschaft**-Objekt in ein **Verein**-Objekt von der Objektleiste aus eintragen wollen, dann gibt es dazu zwei Möglichkeiten. Entweder erzeugen Sie ein neues **Mitgliedschaft**-Objekt auf der Objektleiste, rufen die Methode **beitreten** auf dem **Verein**-Objekt auf und klicken auf das **Mitgliedschaft**-Objekt, um es als Parameter anzugeben; oder Sie rufen die **beitreten**-Methode am **Verein**-Objekt auf und tippen Folgendes in das Parameterfeld des Methodenaufruf-Dialogs:

```
new Mitgliedschaft ("Mitgliedsname ...", Monat, Jahr)
```

Prüfen Sie jedes Mal, wenn Sie ein Objekt eintragen, mit der Methode **anzahlMitglieder**, ob die Methode **beitreten** wirklich in die Sammlung einfügt und ob die Methode **anzahlMitglieder** selbst das richtige Ergebnis liefert.

Wir werden dieses Projekt noch weiter erkunden, indem wir später weitere Übungen dazu durchführen.

Übung 4.43 Zusatzaufgabe. Diese und die folgenden Übungen sind eine Herausforderung, da sie Kenntnisse in Themen voraussetzen, die wir noch nicht explizit behandelt haben. Trotzdem sollten Sie sich daran wagen, wenn Sie das bisher besprochene Material einigermaßen verstanden haben. Wir werden etwas hinzufügen, über das die meisten Musikplayer verfügen: eine „Shuffle“- beziehungsweise „Zufallsabspiel“-Funktion.

Das Paket **java.util** enthält die Klasse **Random**, deren Methode **nextInt** eine positive ganze Zahl innerhalb eines begrenzten Bereichs erzeugt. Schreiben Sie eine Methode in der Klasse **MusikSammlung**, die einen zufälligen Track aus ihrer Liste auswählt und abspielt.

Hinweis: Sie müssen **Random** importieren und ein **Random**-Objekt erzeugen – und zwar entweder direkt in der neuen Methode oder im Konstruktor und anschließend in einem Datenfeld speichern. Lesen Sie die API-Dokumentation für die **Random**-Klasse, um festzustellen, welche der **nextInt**-Methoden Sie wählen müssen. Alternativ können Sie auch das Kapitel 6 lesen, das sich mit der Klasse **Random** befasst.

Übung 4.44 Zusatzaufgabe. Überlegen Sie, wie Sie mehrere Tracks in einer zufälligen Reihenfolge abspielen könnten. Wollen Sie sicherstellen, dass alle Tracks gleich oft gespielt werden oder Lieblingstracks bevorzugen? Inwiefern könnte ein Datenfeld zur Abspielfrequenz in der Klasse **Track** für diese Aufgabe nützlich sein? Diskutieren Sie die verschiedenen Optionen.

Übung 4.45 Zusatzaufgabe. Schreiben Sie eine Methode, um jeden Track der Liste genau einmal in einer Zufallsreihenfolge abzuspielen.

Hinweis: Eine Möglichkeit wäre, die Reihenfolge der Tracks in der Liste – oder besser in einer Kopie der Liste – neu zu mischen und dann die Liste von Anfang bis Ende abzuspielen. Eine andere Möglichkeit wäre, eine Kopie der Liste zu erstellen und dann wiederholt einen zufälligen Track aus der Liste auszuwählen, abzuspielen und von der Liste zu entfernen, bis die Liste leer ist. Versuchen Sie, einen dieser Ansätze zu implementieren. Wenn Sie den ersten Ansatz verfolgen, stellt sich die Frage, wie einfach es ist, die Liste so zu mischen, dass sie tatsächlich in einer neuen Zufallsreihenfolge vorliegt. Gibt es irgendwelche Bibliotheksmethoden, die Ihnen dabei helfen könnten?

4.14 Ein weiteres Beispiel: ein Auktionssystem

In diesem Abschnitt werden wir die Konzepte, die wir in diesem Kapitel neu kennengelernt haben, aus einem anderen Zusammenhang heraus erneut betrachten.

Das Projekt *Auktion* modelliert einen Teil der Funktionalität eines Online-Auktionssystems. Die Grundidee ist, dass eine Auktion aus einer Reihe von Gegenständen besteht, die zum Verkauf angeboten werden. Diese Gegenstände werden *Posten* genannt und jedem Posten wird eine eindeutige Nummer vom Programm zugewiesen. Eine Person, die einen Posten ersteigern möchte, muss Geld dafür bieten. Unsere Auktionen sind etwas anders als sonst üblich, weil alle Posten nur für kurze Zeit angeboten werden.³ Am Ende dieser Frist wird die Auktion geschlossen. Beim Ende einer Auktion bekommt die Person, die das höchste Gebot für einen Posten abgegeben hat, den Zuschlag. Alle Posten, für die nicht geboten wurde, bleiben unverkauft. Unverkaufte Posten können dann beispielsweise in der nächsten Auktion wieder angeboten werden.

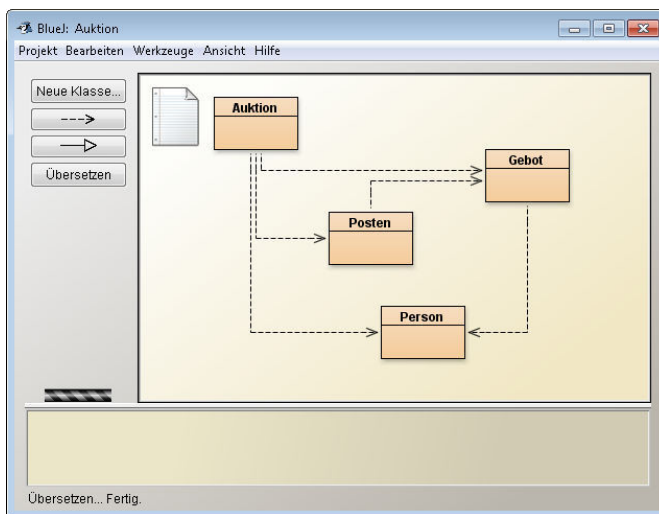


Abbildung 4.6
Die Klassenstruktur
des Projektes *Auktion*.

³ Zur Vereinfachung haben wir den Aspekt der Zeitbegrenzung in den hier betrachteten Klassen nicht implementiert.

Das Projekt *Auktion* enthält folgende Klassen: **Auktion**, **Gebot**, **Posten** und **Person**. Ein genauerer Blick auf das Klassendiagramm für dieses Projekt (Abbildung 4.6) zeigt, dass die Beziehungen zwischen den verschiedenen Klassen ein wenig komplizierter sind als in vorherigen Projekten. Dies hat Auswirkungen auf die Art und Weise, wie auf Informationen während der Auktion zugegriffen wird. Das Diagramm zeigt beispielsweise, dass die **Auktion**-Objekte über alle anderen Objekttypen (**Gebot**, **Posten** und **Person**) informiert sind. **Posten**-Objekte wissen von **Gebot**-Objekten, und **Gebot**-Objekte wissen von **Person**-Objekten. Was uns das Diagramm allerdings nicht verrät, ist, wie genau auf die in einem **Gebot**-Objekt gespeicherten Informationen beispielsweise von einem **Auktion**-Objekt aus zugegriffen wird. Dafür müssen wir uns mit dem Code der jeweiligen Klasse befassen.

4.14.1 Erste Schritte

Zu diesem Zeitpunkt würde es sich anbieten, das Projekt *Auktion* zu öffnen und den Quelltext zu untersuchen, bevor Sie weiterlesen. Neben Bekanntem wie die Verwendung von **ArrayList** und Schleifen werden Ihnen einige Dinge begegnen, die Sie am Anfang noch nicht ganz verstehen werden. Doch dies ist zu erwarten, da wir uns neuen Ideen und Herangehensweisen zuwenden.

Ein **Auktion**-Objekt ist der Ausgangspunkt für unser Projekt. Personen, die etwas verkaufen wollen, geben es mithilfe der Methode **postenAnmelden** in die Auktion, doch sie liefern nur eine String-Beschreibung. Das **Auktion**-Objekt erzeugt daraufhin für jeden eingegebenen Posten ein **Posten**-Objekt. Dies modelliert die Abläufe der realen Welt: Es ist beispielsweise die Auktions-Site und nicht der Verkäufer, die den Posten Postennummern oder Kennungen zuweist. Demzufolge ist ein **Posten**-Objekt die Repräsentation eines zum Verkauf stehenden Postens der Auktions-Site.

Um für Posten mitbieten zu können, müssen sich Interessenten beim Auktionshaus registrieren. In unserem Programm wird ein potenzieller Bieter durch ein **Person**-Objekt repräsentiert. Diese Objekte müssen unabhängig auf der BlueJ-Objektleiste erzeugt werden. In unserem Projekt enthält ein **Person**-Objekt lediglich den Namen der Person. Wenn jemand für einen Posten bieten möchte, ruft er die **gibGebotAb**-Methode des **Auktion**-Objekts auf, unter Angabe der Postennummer des **Person**-Objekts des Bieters und der Höhe des Gebots. Beachten Sie, dass die Postennummer und nicht das **Posten**-Objekt übergeben wird. Die **Posten**-Objekte bleiben Interna des **Auktion**-Objekts und werden von außen über ihre Postennummer referenziert.

Genauso wie das **Auktion**-Objekt ein **Posten**-Objekt erzeugt, wandelt es ein finanzielles Gebot in ein **Gebot**-Objekt um, das den Betrag und die bietende Person speichert. Aus diesem Grund sehen wir im Klassendiagramm eine Verbindung von der **Gebot**-Klasse zu der **Person**-Klasse. Beachten Sie jedoch, dass es keine Verbindung von **Gebot** zu **Posten** gibt. Die Verbindung ist in dem Diagramm andersherum, da ein **Posten** das bisher höchste Gebot für den Posten speichert. Das bedeutet, dass das **Posten**-Objekt das bei ihm gespeicherte **Gebot**-Objekt ersetzt, sobald ein höheres Gebot eingeht.

Was wir hier beschrieben haben, ist eine ziemlich verschachtelte Kette von Objektreferenzen. **Auktion**-Objekte speichern **Posten**-Objekte; jedes **Posten**-Objekt kann

ein **Gebot**-Objekt speichern; jedes **Gebot**-Objekt speichert ein **Person**-Objekt. Solche Ketten kommen in Programmen sehr häufig vor, sodass dieses Projekt eine gute Gelegenheit bietet herauszufinden, wie sie in der Praxis funktionieren.

Übung 4.46 Erzeugen Sie eine Auktion mit einigen Posten, Personen und Geboten. Verwenden Sie dann den Objektinspektor, um die Objektstruktur zu untersuchen. Beginnen Sie mit dem Auktionsobjekt und untersuchen Sie anschließend alle Objektreferenzen, die Ihnen in den Datenfeldern der Objekte begegnen.

Da weder die Klasse **Person** noch die Klasse **Gebot** irgendeine Aktivität im Auktionssystem initiieren, wollen wir hierauf nicht näher eingehen. Das Studium dieser Klassen bleibt dem interessierten Leser als Übung überlassen. Wir werden stattdessen unser Hauptaugenmerk auf den Quelltext der Klassen **Posten** und **Auktion** richten.

4.14.2 Das Schlüsselwort **null**

Aus der obigen Diskussion sollte klar sein, dass ein **Gebot**-Objekt nur erzeugt wird, wenn tatsächlich jemand für einen **Posten** bietet. Das neu erzeugte **Gebot**-Objekt speichert dann die **Person**, die das Gebot abgegeben hat. Das wiederum bedeutet, dass das **Person**-Datenfeld eines **Gebot**-Objekts im **Gebot**-Konstruktor initialisiert werden kann und immer ein gültiges **Person**-Objekt enthält.

Wenn hingegen ein **Posten**-Objekt erzeugt wird, bedeutet dies lediglich, dass der Posten in die Auktion gegeben wurde und noch keinen Bieter gefunden hat. Trotzdem hat dieses Objekt ein Datenfeld **hoechstesGebot** vom Typ **Gebot**, in dem das höchste Gebot für den Posten abgelegt wird. Mit welchem Wert sollte dieses Datenfeld im **Posten**-Konstruktor initialisiert werden?

Was wir benötigen, ist ein Wert für das Datenfeld, aus dem klar hervorgeht, dass es zurzeit „kein Objekt“ gibt, auf das diese Variable verweist. Die Variable ist gewissermaßen „leer“. Hierfür bietet uns Java das Schlüsselwort **null**. Folglich gibt es im Konstruktor von **Posten** die folgende Anweisung:

```
hoechstesGebot = null;
```

Ein sehr wichtiges Prinzip der Programmierung mit Referenzen ist, dass über eine Variable, die den **null**-Wert enthält, keine Methode aufgerufen werden sollte. Die Erklärung liegt auf der Hand: Da Methoden zu Objekten gehören, können wir über eine Variable, die auf kein Objekt verweist, auch keine Methode aufrufen. Das bedeutet, dass wir manchmal eine **if**-Anweisung verwenden müssen, um zu testen, ob eine Variable **null** enthält oder nicht, bevor wir eine Methode auf diese Variable aufrufen können. Wird dieser Test nicht gemacht, dann kommt es sehr häufig zu einem Laufzeitfehler namens **NullPointerException**. Wir werden einige Beispiele für diesen Test sowohl in der Klasse **Posten** als auch in der Klasse **Auktion** sehen.

Konzept

Das Java-Schlüsselwort **null** hat die Bedeutung „kein Objekt“. **null** wird in einer Objektvariablen gespeichert, die nicht auf ein konkretes Objekt verweist. Eine Objektvariable, die nicht explizit initialisiert wurde, enthält als Standardwert **null**.

Genau genommen erhält ein Datenfeld, das vom Typ eines Objekts ist, automatisch den Wert `null`, wenn wir vergessen haben, es zu initialisieren. In diesem speziellen Fall ziehen wir es jedoch vor, die Zuweisung explizit vorzunehmen, sodass der Leser des Codes genau weiß, dass wir bei der Erzeugung eines `Posten`-Objekts davon ausgehen, dass `hoechstesGebot` den Wert `null` hat.

4.14.3 Die Klasse `Posten`

Die Klasse `Posten` speichert eine Beschreibung eines Postens, eine Postennummer und die Details des bisher höchsten Gebots. Die umfangreichste Methode der Klasse ist die Methode `hoeheresGebot` (Listing 4.9). Sie überprüft für ein neues Gebot, ob es höher ist als alle bisherigen Gebote für den Posten. Wenn es höher ist, dann wird es als das aktuell höchste Gebot im Posten vermerkt.

Listing 4.9

Die Behandlung eines Gebots für einen Posten.

```
/**
 * Eine Klasse, die die Posten in einer Auktion modelliert.
 * Ein Posten kann ein einzelner Gegenstand oder eine
 * Zusammenstellung von Gegenständen sein.
 *
 * @author David J. Barnes und Michael Kölling
 * @version 2016.02.29
 */
public class Posten
{
    // das aktuell höchste Gebot für diesen Posten
    private Gebot hoechstesGebot;

    // weitere Datenfelder und Konstruktor hier ausgelassen

    /**
     * Ein Versuch, für diesen Posten zu bieten. Ein erfolgreiches
     * Gebot muss höher sein als alle bisherigen Gebote.
     * @param gebot ein neues Gebot.
     * @return true falls erfolgreich, false sonst
     */
    public boolean hoeheresGebot(Gebot gebot)
    {
        if(hoechstesGebot == null) {
            // es gibt kein älteres Gebot
            hoechstesGebot = gebot;
            return true;
        }
        else if (gebot.gibHoehe() > hoechstesGebot.gibHoehe()){
            // das Gebot ist besser als das letzte
            hoechstesGebot = gebot;
            return true;
        }
        else {
            // das Gebot ist nicht besser
            return false;
        }
    }

    // weitere Methoden hier ausgelassen
}
```

Hier überprüfen wir zuerst, ob das Gebot tatsächlich das höchste ist. Das ist entweder der Fall, wenn es vorher noch kein Gebot gegeben hat, oder wenn das bisher höchste Gebot niedriger ist als das gegebene. Der erste Teil der Prüfung ist der folgende Vergleich:

```
hoechstesGebot == null
```

Dieser Ausdruck testet, ob die Variable `hoechstesGebot` momentan eine Referenz auf ein Objekt enthält. Wie bereits im vorherigen Abschnitt beschrieben, enthält das Datenfeld `hoechstesGebot` so lange den Wert `null`, bis ein Gebot für den Posten abgegeben wurde. Wenn es noch `null` ist, dann ist dies das erste Gebot für einen bestimmten Posten und somit auch das höchste. Ist es nicht `null`, dann müssen wir den Wert mit dem neuen Gebot vergleichen. Beachten Sie, dass ein Fehlschlagen der ersten Prüfung uns einige sehr nützliche Informationen liefert: Wir wissen jetzt ganz sicher, dass `hoechstesGebot` nicht `null` ist, sodass wir ohne Bedenken eine Methode darüber aufrufen können. Wir müssen in dieser zweiten Bedingung nicht erneut auf `null` testen. Die Werte der beiden Gebote zu vergleichen, erlaubt uns, ein höheres neues Gebot zu wählen oder das neue Gebot zu verwerfen, wenn es nicht besser ist.

4.14.4 Die Klasse Auktion

In der Klasse `Auktion` (Listing 4.10) sehen wir eine weitere Verwendung der Konzepte von `ArrayList` und `for-each`-Schleifen, die wir in diesem Kapitel bereits diskutiert haben.

```
import java.util.ArrayList;

/**
 * Ein einfaches Modell einer Auktion.
 * Ein Exemplar dieser Klasse hält eine Liste von Posten,
 * die beliebig lang werden kann.
 *
 * @author David J. Barnes und Michael Kölling
 * @version 2016.02.29
 */
public class Auktion
{
    // eine Liste der Posten dieser Auktion
    private ArrayList<Posten> postenliste;
    // die Nummer, die an den nächsten Posten vergeben wird,
    // der für diese Auktion angemeldet wird
    private int naechstePostennummer;

    /**
     * Erzeuge eine neue Auktion.
     */
    public Auktion()
    {
        postenliste = new ArrayList<>();
        naechstePostennummer = 1;
    }

    /**
     * Melde einen Posten für diese Auktion an.
     * @param beschreibung die Beschreibung des Postens
     */
    public void postenAnmelden(String beschreibung)
    {
        postenliste.add(new Posten(naechstePostennummer, beschreibung));
        naechstePostennummer++;
    }

    /**
     * Zeige die komplette Liste der Posten dieser Auktion.
     */
    public void zeigePostenliste()
    {
        for (Posten posten : postenliste) {
            System.out.println(posten.toString());
        }
    }
}
```

Listing 4.10
Die Klasse `Auktion`.

```

/**
 * Gebe für einen Posten ein Angebot ab.
 * Es erfolgt eine Ausgabe, ob das Gebot erfolgreich war oder nicht.
 * @param postennummer der Posten, für den geboten wird
 * @param bieter die Person, die bietet
 * @param betrag die Höhe des Gebots
 */
public void gibGebotAb(int postennummer, Person bieter, long betrag)
{
    Posten gewaelterPosten = gibPosten(postennummer);
    if(gewaelterPosten != null) {
        Gebot gebot = new Gebot(bieter, betrag);
        boolean erfolgreich = gewaelterPosten.hoeheresGebot(gebot);
        if(erfolgreich) {
            System.out.println("Das Gebot für Posten Nummer " +
                               postennummer + " war erfolgreich.");
        }
        else {
            // Melden, welches Gebot höher ist.
            Gebot hoechstesGebot = gewaelterPosten.gibHoechstesGebot();
            System.out.println("Posten Nummer " + postennummer +
                               ": Gebot steht bereits bei " +
                               hoechstesGebot.gibHoehe());
        }
    }
}

/**
 * Liefere den Posten mit der angegebenen Nummer.
 * Liefere 'null', wenn ein Posten mit dieser Nummer
 * nicht existiert.
 * @param nummer die Nummer des Postens, der geliefert
 *                werden soll
 */
public Posten gibPosten(int nummer)
{
    if((nummer >= 1) && (nummer < naechstePostennummer)) {
        // die Nummer scheint gültig zu sein.
        Posten gewaelterPosten = postenliste.get(nummer-1);
        // ein Sicherheitscheck, ob wir auch den richtigen
        // Posten haben:
        if(gewaelterPosten.gibNummer() != nummer) {
            System.out.println("Interner Fehler: Posten Nummer " +
                               gewaelterPosten.gibNummer() +
                               " wurde geliefert anstelle von " +
                               nummer);
        }
        return gewaelterPosten;
    }
    else {
        System.out.println("Einen Posten mit der Nummer: " + nummer +
                           " gibt es nicht.");
        return null;
    }
}

```

Das Datenfeld **posten** ist eine **ArrayList**, die die Posten hält, die in dieser Auktion angeboten werden. Posten werden in die Auktion eingebracht, indem der Methode **postenAnmelden** eine einfache Beschreibung übergeben wird. Ein neuer Posten wird erzeugt, indem diese Beschreibung zusammen mit einer eindeutigen Postennummer an den Konstruktor der Klasse **Posten** übergeben wird. Das neue **Posten**-Objekt wird dann in die Sammlung eingefügt. Die folgenden Abschnitte diskutieren einige zusätzliche, häufig vorkommende Besonderheiten anhand der Klasse **Auktion**.

4.14.5 Anonyme Objekte

Die Methode **postenAnmelden** in der Klasse **Auktion** demonstriert ein häufig auftretendes Programmiermuster – anonyme Objekte. Wir erkennen es in der folgenden Anweisung:

```
postenliste.add(new Posten(naechstePostennummer, beschreibung));
```

Hier nehmen wir zwei Dinge vor:

- Wir erzeugen ein neues **Posten**-Objekt und
- wir übergeben dieses neue Objekt an die Methode **add** der **ArrayList**.

Wir hätten das Gleiche in zwei Zeilen schreiben können, um die beiden Schritte expliziter zu machen:

```
Posten neuerPosten = new Posten(naechstePostennummer, beschreibung);
postenliste.add(neuerPosten);
```

Beide Versionen sind äquivalent, aber wenn wir für die Variable **neuerPosten** keine sonstige Verwendung haben, dann vermeidet die Originalfassung die Deklaration einer Variablen mit so begrenztem Nutzen. Wir erzeugen also ein anonymes Objekt – ein Objekt ohne Bezeichnung –, indem wir es direkt an die Methode übergeben, die es verwendet.

Übung 4.47 Die Methode **gibGebotAb** umfasst die folgenden zwei Anweisungen:

```
Gebot gebot = new Gebot(bieter, betrag);
boolean erfolgreich = gewaehlterPosten.hoeheresGebot(gebot);
```

Die Variable **gebot** wird hier nur als Platzhalter für das neu erzeugte **Gebot**-Objekt verwendet, bevor es direkt an die Methode **hoeheresGebot** übergeben wird. Schreiben Sie diese Anweisungen so um, dass Sie auf die Variable **gebot** verzichten können. Verwenden Sie dazu wie in der Methode **postenAnmelden** ein anonymes Objekt.

4.14.6 Methodenaufrufe verketteten

In unserer Einführung des *Auktion*-Projektes haben wir auf eine Kette von Objektreferenzen hingewiesen: **Auktion**-Objekte speichern **Posten**-Objekte; jedes **Posten**-Objekt kann ein **Gebot**-Objekt speichern; jedes **Gebot**-Objekt speichert ein **Person**-Objekt. Wenn das **Auktion**-Objekt identifizieren muss, wer gegenwärtig das höchste Gebot für einen Posten abgegeben hat, dann muss es das **Posten**-Objekt auffordern, das **Gebot**-Objekt für diesen Posten zurückzuliefern, dann das **Gebot**-Objekt nach dem **Person**-Objekt fragen, von dem das Gebot stammt, und schließlich dieses **Person**-Objekt nach seinem Namen fragen.

Wenn wir die Möglichkeit von **null**-Objektreferenzen außer Acht lassen, könnten wir in etwa folgende Folge von Anweisungen haben, um den Namen eines Bieters auszugeben:

```
Gebot gebot = posten.gibHoechstesGebot();
Person bieter = gebot.gibBieter();
String name = bieter.gibName();
System.out.println(name);
```

Da die Variablen **gebot**, **bieter** und **name** hier nur als Zwischenspeicher verwendet werden, um an den Namen des Bieters zu gelangen, kommt es häufig vor, dass solche Anweisungsfolgen durch die Verwendung von anonymen Objektreferenzen komprimiert werden. Wir erhalten beispielsweise den gleichen Effekt mit folgender Zeile:

```
System.out.println(posten.gibHoechstesGebot().gibBieter().gibName());
```

Das sieht aus, als ob Methoden Methoden aufrufen, aber so darf es nicht gelesen werden. In Anbetracht der Tatsache, dass diese beiden Quelltextabschnitte gleich sind, muss die Kette der Methodenaufrufe streng von links nach rechts gelesen werden:

```
posten.gibHoechstesGebot().gibBieter().gibName()
```

Der Aufruf von **gibHoechstesGebot** liefert ein anonymes **Gebot**-Objekt, auf das anschließend die Methode **gibBieter** aufgerufen wird. Entsprechend liefert **gibBieter** ein anonymes **Person**-Objekt, sodass **gibName** auf diese Person aufgerufen wird.

Solche Ketten von Methodenaufrufen können kompliziert aussehen, doch sie lassen sich entwirren, wenn Sie die zugrunde liegenden Regeln kennen. Selbst wenn Sie sich bei Ihrem eigenen Code gegen diese kompakte Schreibweise entscheiden, sollten Sie wissen, wie er zu lesen ist, da Sie in den Quelltexten anderer Programmierer mit Sicherheit darauf treffen werden.

Übung 4.48 Fügen Sie der Klasse **Auktion** eine Methode **beenden** hinzu. Diese sollte die Sammlung der Posten durchlaufen und die Details aller Posten ausgeben. Verwenden Sie dazu eine **for-each**-Schleife. Posten mit mindestens einem Gebot sollten als verkauft angesehen werden, sodass Sie nach **Posten**-Objekten Ausschau halten, deren Datenfeld **hoechstesGebot** nicht **null** ist. Verwenden Sie eine lokale Variable in der Schleife, um den Wert zu speichern, der durch Aufrufe der Methode **gibHoechstesGebot** zurückgeliefert wird, und testen Sie diese Variable auf den Wert **null**.

Bei verkauften Posten sollten der Name des erfolgreichen Bieters und das Höchstgebot ausgegeben werden, bei unverkauften Posten eine kurze Meldung, die dies deutlich macht.

Übung 4.49 Fügen Sie der Klasse **Auktion** eine Methode **gibUnverkaufte** hinzu, die die folgende Signatur hat:

```
public ArrayList<Posten> gibUnverkaufte();
```

Diese Methode soll über das Datenfeld **posten** iterieren und unverkaufte Posten in einer neuen lokalen **ArrayList**-Variablen ablegen. Sie suchen nach **Posten**-Objekten, deren Datenfeld **hoechstesGebot** den Wert **null** hat. Am Ende soll diese Liste als Ergebnis der Methode zurückgegeben werden.

Übung 4.50 Nehmen Sie an, die Klasse **Auktion** enthielte eine Methode, mit der ein Posten aus einer Auktion herausgenommen werden kann. Wenn Sie weiterhin annehmen, dass bei den übrig gebliebenen Posten das Datenfeld **postennummer** nicht verändert wird, wenn ein Posten entfernt wird: Wie würde sich die Möglichkeit zum Entfernen von Posten auf die Methode **gibPosten** auswirken?

Übung 4.51 Ändern Sie die Methode **gibPosten** so ab, dass diese sich nicht darauf verlässt, dass ein Posten mit einer bestimmten Nummer an Indexposition (**nummer-1**) in der Sammlung gespeichert ist. Wenn zum Beispiel Postennummer 2 entfernt wurde, dann wird Postennummer 3 von Indexposition 2 auf Indexposition 1 verschoben und alle höheren Postennummern werden ebenfalls um eine Indexposition verschoben. Sie können jedoch weiterhin davon ausgehen, dass die Posten nach aufsteigender Postennummer abgelegt werden.

Übung 4.52 Fügen Sie der Klasse **Auktion** eine Methode **entfernePosten** mit folgendem Kopf hinzu:

```
/**
 * Entferne den Posten mit der gegebenen Postennummer.
 * @param nummer die Nummer des Postens, der entfernt werden soll.
 * @return den Posten mit der gegebenen Nummer, oder null, falls
 * es einen solchen Posten nicht gibt.
 */
public Posten entfernePosten(int nummer)
```

Diese Methode soll nicht davon ausgehen, dass ein Posten mit einer bestimmten Nummer an einer bestimmten Position innerhalb der Liste abgelegt ist.

Übung 4.53 Die Klasse **ArrayList** ist im Paket **java.util** definiert. Dieses Paket enthält auch eine Klasse **LinkedList**. Versuchen Sie, alles über die Klasse **LinkedList** herauszufinden, und vergleichen Sie ihre Methoden mit denen von **ArrayList**. Über welche Methoden verfügen beide Klassen und in welchen unterscheiden sie sich?

4.14.7 Der Umgang mit Sammlungen

Die Sammlungsklasse **ArrayList** (und andere ähnliche Klassen) ist ein wichtiges Hilfsmittel bei der Programmierung, weil viele Programmierprobleme flexible Sammlungen von Objekten betreffen. Bevor Sie sich den nächsten Kapiteln zuwenden, sollten Sie sich mit ihrem Umgang vertraut machen. Die folgenden Übungen ermöglichen Ihnen dies.

Übung 4.54 Arbeiten Sie weiter mit dem Projekt *Verein* aus **Übung 4.40**. Definieren Sie in der Klasse **Verein** eine Methode, die folgendermaßen beschrieben ist:

```
/**
 * Liefere die Anzahl der Mitglieder, die im angegebenen Monat
 * Mitglied geworden sind.
 * @param monat der Monat des Beitritts.
 * @return die Anzahl der Mitglieder, die im Monat beigetreten sind.
 */
public int beigetretenInMonat(int monat)
```

Wenn der Parameter **monat** außerhalb des gültigen Bereichs von 1 bis 12 liegt, dann gibt die Methode eine Fehlermeldung aus und liefert 0 zurück.

Übung 4.55 Definieren Sie in der Klasse **Verein** eine Methode, die folgendermaßen beschrieben ist:

```
/**
 * Entferne alle Mitglieder aus der Sammlung des Vereins, die im
 * angegebenen Monat Mitglied geworden sind, und liefere sie in
 * einem getrennten Sammlungsobjekt zurück.
 * @param monat der Monat des Beitritts.
 * @param jahr das Jahr des Beitritts.
 * @return die neuen Mitglieder des gegebenen Monats und Jahres.
 */
public ArrayList<Mitgliedschaft> entfernen(int monat, int jahr)
```

Wenn der Parameter **monat** außerhalb des gültigen Bereichs von 1 bis 12 liegt, dann gibt die Methode eine Fehlermeldung aus und liefert ein Sammlungsobjekt ohne Objekte zurück.

Hinweis: Die Methode **entfernen** ist deutlich schwieriger zu programmieren als alle anderen Methoden dieser Klasse.

Übung 4.56 Öffnen Sie das Projekt *Artikelbestand* und vervollständigen Sie die Klasse **Bestandsverwalter** in dieser und den folgenden Übungen. Der **Bestandsverwalter** verwendet eine **ArrayList**, um Artikel zu speichern. Seine Methode **neuerArtikel** führt bereits einen neuen Artikel in die Verwaltung ein, aber die folgenden Methoden müssen noch vervollständigt werden: **aufnehmen**, **findeArtikel**, **alleArtikelAnzeigen** und **mengeImBestand**.

Jeder Artikel, der von der Firma verkauft wird, ist durch ein Objekt der Klasse **Artikel** repräsentiert, das den Namen, die Nummer und den aktuellen Bestand dieses Artikels im Lager hält. Die Klasse **Artikel** definiert eine Methode **erhoeheBestand**, mit der der Bestand erhöht werden kann. Die Methode **verkaufeEinen** vermerkt, wenn ein einzelner Artikel verkauft wurde, indem der Bestand um 1 reduziert wird. Die Klasse **Artikel** ist vorgegeben und Sie sollten diese nicht ändern müssen. Beginnen Sie, indem Sie die Methode **alleArtikelAnzeigen** implementieren und auf diese Weise feststellen, ob Sie über alle Elemente der Sammlung iterieren können. Geben Sie einfach die Details jedes gelieferten Artikels aus, indem Sie seine **toString**-Methode aufrufen.

Übung 4.57 Implementieren Sie die Methode `findeArtikel`. Diese sollte die Artikelsammlung nach einem Artikel mit passender Nummer durchsuchen. Wenn ein passender Artikel gefunden wird, sollte er als Ergebnis des Aufrufs zurückgeliefert werden. Wenn kein Artikel gefunden werden kann, wird `null` geliefert.

Diese Methode unterscheidet sich von `alleArtikelAnzeigen` dadurch, dass nicht notwendigerweise alle Elemente der Sammlung untersucht werden müssen, bevor ein Artikel gefunden wird. Wenn die Artikelnummer beispielsweise gleich auf den ersten Artikel in der Sammlung passt, dann kann die Iteration abgebrochen und der Artikel geliefert werden. Andererseits kann es passieren, dass die gesuchte Artikelnummer gar nicht bekannt ist. In diesem Fall wird die gesamte Sammlung untersucht, ohne dass ein Artikel gefunden wird. Dann sollte `null` geliefert werden.

Wenn Sie nach einem passenden Artikel suchen, werden Sie die Methode `gibNummer` auf einem Artikel aufrufen müssen.

Übung 4.58 Implementieren Sie die Methode `mengeImBestand`. Diese sollte einen Artikel mit passender Nummer in der Sammlung finden und die aktuell vorhandene Menge als Methodenergebnis zurückliefern. Wenn kein Artikel mit passender Nummer gefunden wird, soll 0 geliefert werden. Dies ist relativ einfach zu implementieren, sobald die Methode `findeArtikel` zur Verfügung steht. `mengeImBestand` könnte beispielsweise `findeArtikel` für die Suche verwenden und dann auf dem Ergebnis die Methode `gibBestand` aufrufen. Beachten Sie aber den Fall, wenn ein Artikel nicht gefunden werden kann.

Übung 4.59 Implementieren Sie die Methode `aufnehmen` nach einem ähnlichen Muster wie bei `mengeImBestand`. Sie sollte den Artikel mit der gegebenen Nummer finden und dann auf diesem die Methode `erhoeheBestand` aufrufen.

Übung 4.60 Zusatzaufgabe. Implementieren Sie eine Methode im `Bestandsverwalter`, die die Details aller Artikel ausgibt, deren Bestand unter einen bestimmten Level gesunken ist (der als Parameter an die Methode übergeben wird).

Verändern Sie die Methode `neuerArtikel` so, dass ein neuer Artikel nicht eingefügt werden kann, wenn bereits ein Artikel mit dieser Nummer existiert.

Fügen Sie beim `Bestandsverwalter` eine Methode ein, mit deren Hilfe ein Artikel nicht über seine Nummer, sondern über seinen Namen gefunden werden kann:

```
public Artikel findeArtikel(String name)
```

Für die Lösung müssen Sie wissen, dass zwei `String`-Objekte `s1` und `s2` mit dem booleschen Ausdruck

```
s1.equals(s2)
```

auf Gleichheit getestet werden können. Mehr zum Thema Vergleichen von Strings finden Sie in Kapitel 6.



Zusammenfassung

Wir haben gesehen, dass uns Klassen wie **ArrayList** auf einfache Weise ermöglichen, Sammlungen mit einer beliebigen Anzahl von Elementen zu erzeugen. Die Java-Bibliothek enthält weitere Sammlungen dieser Art, und wir werden uns einige von ihnen in Kapitel 6 ansehen. Sie werden feststellen, dass ein souveränes Umgehen mit Sammlungen eine wichtige Fähigkeit für das Entwickeln interessanter Programme ist. Es gibt praktisch keine Anwendung, die nicht in irgendeiner Form Sammlungen verwendet.

Beim Umgang mit Sammlungen entsteht der Bedarf, über alle Elemente einer Sammlung zu iterieren, um von jedem Objekt Gebrauch machen zu können. Zu diesem Zweck haben wir uns die Benutzung von Schleifen und Iteratoren angesehen.

Schleifen sind ebenfalls ein fundamentales Konzept der Programmierung, das Sie von nun an in allen weiteren Projekten verwenden werden. Stellen Sie sicher, dass Sie den Umgang mit Schleifen ausreichend geübt haben – ohne Schleifen werden Sie nicht sehr weit kommen. Um zu entscheiden, welche Art von Schleife in welcher Situation angebracht ist, ist es nützlich zu wissen, ob die Aufgabe mit einer bestimmten oder unbestimmten Iteration verbunden ist. Ist die Anzahl der benötigten Durchläufe bekannt oder nicht?

Am Rande haben wir auch die Klassenbibliothek von Java erwähnt – eine große Sammlung nützlicher Klassen, die wir einsetzen können, um unsere eigenen Klassen leistungsfähiger zu machen. Wir sollten diese Bibliothek ausführlicher untersuchen, um zu sehen, welche weiteren Klassen wir kennen sollten. Die Benutzung der Bibliothek ist deshalb Thema eines späteren Kapitels.

NEUE BEGRIFFE IN DIESEM KAPITEL

Sammlung, Iterator, **for-each**-Schleife, **while**-Schleife, Index, **import**-Anweisung, Bibliothek, Paket, anonymes Objekt, bestimmte Iteration, unbestimmte Iteration

Zusammenfassung der Konzepte

- **Sammlung** Sammlungsobjekte sind Objekte, die eine beliebige Anzahl anderer Objekte enthalten können.
- **Schleife** Eine Schleife kann benutzt werden, um einen Block von Anweisungen wiederholt ausführen zu lassen, ohne dass die Anweisungen mehrfach in den Quelltext geschrieben werden müssen.
- **Iterator** Ein Iterator ist ein Objekt, mit dessen Hilfe über alle Elemente einer Sammlung iteriert werden kann.
- **null** Das Java-Schlüsselwort `null` hat die Bedeutung „kein Objekt“. `null` wird in einer Objektvariablen gespeichert, die nicht auf ein konkretes Objekt verweist. Eine Objektvariable, die nicht explizit initialisiert wurde, enthält als Standardwert `null`.



Übung 4.61 Java bietet einen weiteren Schleifentyp an: die *do-while-Schleife*. Finden Sie heraus, wie diese Schleife funktioniert, und beschreiben Sie es. Schreiben Sie eine *do-while*-Schleife, die die Zahlen von 1 bis 10 ausgibt. Um etwas über diese Schleife nachzulesen, sollten Sie eine Sprachbeschreibung von Java benutzen (beispielsweise <http://download.oracle.com/javase/1/tutorial/java/nutsandbolts/> im Abschnitt *Control Flow Statements*).

Übung 4.62 Schreiben Sie die Methode `alleDateienAusgeben` in der Klasse `MusikSammlung` des Projekts *Musiksammlung-v3* so um, dass eine *do-while*-Schleife anstelle einer *for-each*-Schleife verwendet wird. Testen Sie Ihre Lösung sorgfältig. Funktioniert sie auch, wenn die Sammlung leer ist?

Übung 4.63 *Zusatzaufgabe*. Schreiben Sie die Methode `findeErste` in der Klasse `MusikSammlung` des Projekts *Musiksammlung-v4* so um, dass eine *do-while*-Schleife anstelle einer *while*-Schleife verwendet wird. Testen Sie Ihre Lösung sorgfältig. Starten Sie Suchläufe, die erfolgreich sind, und solche, die fehlschlagen. Starten Sie Suchläufe, in denen die gesuchte Datei die erste bzw. die letzte in der Liste ist.

Übung 4.64 Informieren Sie sich über die *switch-case-Anweisung* von Java. Welchen Zweck hat sie? Wie wird sie benutzt? Schreiben Sie ein Beispiel. (Dies ist ebenfalls eine *Anweisung zur Steuerung des Kontrollflusses (control flow)*, daher werden Sie Informationen an ähnlichen Stellen finden wie für die *do-while*-Schleife.)



KAPITEL

5

Funktionale Verarbeitung von Sammlungen (fortgeschrittene Konzepte)

Lernziele

Zentrale Konzepte in diesem Kapitel: Funktionale Programmierung, Lambda-Ausdrücke, Streams, Pipelines

Java-Konstrukte in diesem Kapitel: Stream, Lambda-Ausdruck, `filter`, `map`, `reduce`, `->`

In diesem Kapitel führen wir die ersten der fortgeschrittenen Konzepte ein. Abschnitte dieser Art (es wird später noch einige mehr geben) können beim ersten Lesen getrost übersprungen werden. Wir werden in diesem Kapitel einige alternative Techniken besprechen, mit denen dieselben Ziele erreicht werden können wie mit den Methoden, die wir in den vorigen Kapiteln diskutiert haben. Diese Techniken sind in Java erst ab Version 8 integriert.

5.1 Ein neuer Blick auf die Themen von Kapitel 4

In Kapitel 4 haben wir uns Techniken angesehen, mit denen Sammlungen von Objekten verarbeitet werden konnten. Der Umgang mit Sammlungen ist eine der fundamentalsten Techniken im Programmieren; fast alle Computerprogramme verarbeiten in irgendeiner Art Sammlungen und wir werden in diesem Buch noch sehr vielen weiteren begegnen. Es ist eine entscheidende Fähigkeit, dies gut zu beherrschen.

Das Programmierenlernen hinsichtlich Sammlungen ist jedoch in den letzten Jahren durch die Einführungen von neuen Programmier-Techniken zu den großen Programmiersprachen komplizierter geworden. Diese Techniken können zwar Ihr *Programm* vereinfachen, aber sie zu *lernen* ist aufwendiger – es gibt jetzt schlicht mehrere unterschiedliche Konstrukte, die zu meistern sind.

Die „neuen“ Techniken sind nicht wirklich neu – sie sind nur neu innerhalb dieser bestimmten Sprachart. Es gab immer unterschiedliche Arten von Sprachen: Java ist beispielsweise eine *imperative objektorientierte* Sprache, und der Stil zur Verarbeitung von Sammlungen, den wir in Kapitel 4 vorgestellt haben, ist eine typische imperative Herangehensweise.